Universidad Nacional Autónoma de México

Facultad de Ingeniería

Ingeniería en Computación

Proyecto 1: Desarrollo de una FFT de 1024 puntos mediante mariposas

Presentan:

Becerril Vélez Liliana Marlene
López Campillo Francisco Daniel

Asignatura:
Procesamiento Digital de Señales

Ciudad de México, Abril de 2026

Índice

1. Introduction **3**

2. Planteamiento del problema **4**
2.1. Transformada Discreta de Fourier (DFT) 4
2.2. Análisis de Complejidad y Limitaciones Computacionales 4
2.3. Descomposición Radix-2 y Estructura de Etapas 5
2.4. La Unidad Operativa: La Mariposa (Butterfly) 5
2.5. Objetivos del Planteamiento 5

3. Desarrollo por mariposas **6**
3.1. Descripción de la Operación Básica de Mariposa (Radix-2) 6
3.2. Agrupación de Muestras y Organización por Etapas 7
3.3. Uso de los Factores de Giro (Twiddle Factors) 7
3.4. Flujo de Datos: De la Entrada a la Salida 7

4. Estructura de las 10 etapas **8**
4.1. Determinación del Número de Etapas (ν) 8
4.2. Densidad de Operaciones: Mariposas por Etapa 8
4.3. Dinámica de Distancias entre Índices (Saltos) 9
4.4. Distribución de Factores de Giro por Etapa 9

5. Ordenamiento de datos **9**
5.1. Reordenamiento por Inversión de Bits (*Bit-Reversal*) 10
5.2. Decimación en Frecuencia (DIF) 10
5.3. Localización del Reordenamiento según el Esquema 10

6. Diagrama o esquema **11**
6.1. Cálculo Detallado de la Etapa 1 12
6.1.1. Cálculos de los primeros nodos 13
6.1.2. Cálculos de los nodos finales 13
6.1.3. Generalización de la Etapa 1 13
6.2. Cálculo Detallado de la Etapa 2 14
6.2.1. Cálculos de los primeros nodos 14
6.2.2. Cálculos de los nodos finales 15
6.2.3. Generalización de la Etapa 2 15
6.3. Cálculo Detallado de la Etapa 3 16
6.3.1. Cálculos del primer bloque de 8 puntos 17
6.3.2. Cálculos de los nodos finales 17
6.3.3. Análisis de los Factores de Giro en la Etapa 3 17
6.4. Cálculo Detallado de la Etapa 4 18
6.4.1. Cálculo de la primera mariposa del sistema 19
6.4.2. Cálculo de la última mariposa del sistema 19

6.4.3. Análisis de los Factores de Giro en la Etapa 4 19
6.5. Cálculo Detallado de la Etapa 5 20
6.5.1. Cálculo de la mariposa superior (Bloque 1) 21
6.5.2. Cálculo de la mariposa inferior (Último Bloque) 21
6.5.3. Análisis de los Factores de Giro en la Etapa 5 21
6.6. Cálculo Detallado de la Etapa 6 22
6.6.1. Cálculo de la mariposa superior (Bloque 1) 23
6.6.2. Cálculo de la mariposa inferior (Último Bloque) 23
6.6.3. Análisis de los Factores de Giro en la Etapa 6 23
6.7. Cálculo Detallado de la Etapa 7 24
6.7.1. Cálculo de la mariposa superior (Bloque 1) 25
6.7.2. Cálculo de la mariposa inferior (Último Bloque) 25
6.7.3. Análisis de los Factores de Giro en la Etapa 7 25
6.8. Cálculo Detallado de la Etapa 8 26
6.8.1. Cálculo de la mariposa superior (Bloque 1) 27
6.8.2. Cálculo de la mariposa inferior (Último Bloque) 27
6.8.3. Análisis de los Factores de Giro en la Etapa 8 27
6.9. Cálculo Detallado de la Etapa 9 28
6.9.1. Cálculo de la mariposa superior (Bloque 1) 29
6.9.2. Cálculo de la mariposa inferior (Bloque 2) 29
6.9.3. Análisis de los Factores de Giro en la Etapa 9 29
6.10. Cálculo Detallado de la Etapa 10 (Etapa Final) 30
6.10.1. Cálculo de la mariposa superior 31
6.10.2. Cálculo de la mariposa inferior 31
6.10.3. Análisis de los Factores de Giro en la Etapa 10 31
6.10.4. Resultado Final y Salida del Sistema 31

7. Codificación **32**
7.1. Generación de la Señal de Prueba (Aleatoria) 32
7.2. Algoritmo de Reordenamiento: Bit-Reversal 33
7.3. Función de Procesamiento por Etapas: La Unidad de Mariposa 34
7.4. Ejecución en Cascada de las 10 Etapas 35
7.5. Visualización de la Señal de Entrada en el Dominio del Tiempo 35
7.6. Análisis de la Señal Aleatoria en el Dominio del Tiempo 36
7.7. Análisis del Espectro de Magnitud (Salida Etapa 10) 37
7.8. Validación Numérica mediante la Biblioteca Numpy 38
7.9. Validación Espectral mediante una Senoidal Pura 38
7.10. Procesamiento de Señales de Audio Externas (YouTube) 40
7.11. Validación Final: Análisis Espectral de un Instrumento Real 44

8. Complejidad computacional **46**

9. Conclusiones **46**

1. Introduction

El estudio del análisis de señales tiene sus inicios en el trabajo de Jean-Baptiste Joseph Fourier, un matemático y físico francés que, a principios del siglo XIX, postuló una idea revolucionaria: cualquier función periódica puede ser representada como una suma infinita de senos y cosenos. Aunque originalmente Fourier desarrolló estas ideas para resolver problemas de conducción de calor, su legado sentó las bases de lo que hoy conocemos como la Transformada de Fourier. Con la llegada de la era digital y la necesidad de procesar señales mediante computadoras es que surge la Transformada Discreta de Fourier (DFT). Esta variante permitió traducir el concepto continuo de Fourier a un entorno de muestras finitas y discretas, convirtiéndose en la herramienta matemática estándar para descomponer señales digitales en sus componentes de frecuencia.

A pesar de la potencia teórica de la DFT, su implementación práctica era sumamente ineficiente debido a la enorme cantidad de cálculos requeridos. No fue sino hasta 1965 cuando James Cooley y John Tukey publicaron un algoritmo que revolucionaría el procesamiento digital de señales: la Transformada Rápida de Fourier (FFT). Aunque se descubrió que matemáticos como Gauss ya habían trabajado con ideas similares siglos atrás, el algoritmo de Cooley-Tukey permitió que tareas que antes tomaban horas en las computadoras de la época se realizaran en fracciones de segundo, impulsando el desarrollo de las telecomunicaciones, la medicina y la inteligencia artificial moderna.

La diferencia fundamental entre la DFT y la FFT no radica en el resultado final, sino en el método de ejecución. Mientras que la DFT es la definición matemática formal que describe cómo transformar los datos del dominio del tiempo al de la frecuencia punto por punto, la FFT es un algoritmo optimizado diseñado para calcular exactamente ese mismo resultado de una manera mucho más veloz. En otras palabras, la DFT es lo que queremos obtener, y la FFT es cómo lo obtenemos mediante una arquitectura de cómputo inteligente.

La reducción de la complejidad computacional en la FFT se logra gracias a la estrategia de divide y vencerás, específicamente mediante la estructura de operaciones tipo mariposa. En una DFT estándar, para procesar N puntos, el sistema debe realizar N^2 multiplicaciones complejas, ya que cada punto de salida depende de cada punto de entrada. Sin embargo, la FFT aprovecha las propiedades de simetría y periodicidad de los factores de giro complejos en el círculo unitario. Al descomponer la serie original de N puntos en sub-series más pequeñas y reutilizar los resultados de cálculos intermedios a través de las mariposas, la complejidad se reduce de forma drástica a una escala de $O(N \log_2 N)$. Para un sistema de 1024 puntos como el que se trabajará más adelante, esto significa pasar de más de un millón de operaciones a poco más de cinco mil, permitiendo el procesamiento de señales en tiempo real que define la tecnología actual.

2. Planteamiento del problema

El presente proyecto aborda la problemática de la transformación de señales del dominio temporal al dominio frecuencial con alta eficiencia computacional. El análisis de una señal discreta de $N = 1024$ puntos requiere una arquitectura lógica capaz de procesar grandes volúmenes de datos en tiempos de ejecución reducidos, lo cual justifica la transición de la definición formal de Fourier hacia un algoritmo de optimización de base 2.

2.1. Transformada Discreta de Fourier (DFT)

Considerando una secuencia de datos de entrada $x[n]$ obtenida mediante el muestreo de una señal analógica a intervalos regulares. La representación en el dominio de la frecuencia, denotada como $X[k]$, se obtiene mediante la proyección de la señal sobre una base de funciones ortogonales complejas. Para un sistema de N puntos, la expresión matemática fundamental es:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad \text{para } 0 \leq k < N \tag{1}$$

Donde el núcleo de la transformación (*kernel*), conocido como factor de giro o *twiddle factor* W_N, se define mediante la relación exponencial compleja:

$$W_N = e^{-j\frac{2\pi}{N}} \tag{2}$$

Al expandir esta expresión mediante la identidad de Euler, observamos que cada componente k del espectro de frecuencia requiere la evaluación de una serie de productos escalares complejos:

$$W_N^{nk} = \cos\left(\frac{2\pi nk}{N}\right) - j\sin\left(\frac{2\pi nk}{N}\right) \tag{3}$$

2.2. Análisis de Complejidad y Limitaciones Computacionales

En una implementación directa de la ecuación (1), el cálculo de cada coeficiente $X[k]$ implica realizar N multiplicaciones complejas y $N - 1$ adiciones complejas. Dado que el espectro completo consta de N coeficientes, la complejidad algorítmica resultante es de clase $\mathcal{O}(N^2)$.

Para el caso específico de este desarrollo, donde $N = 1024$, el costo computacional se cuantifica de la siguiente manera:

- **Multiplicaciones complejas:** $N^2 = (1024)^2 = 1,048,576$ operaciones.
- **Adiciones complejas:** $N(N-1) = 1024 \times 1023 = 1,047,552$ operaciones.

Esta carga operativa resulta prohibitiva para aplicaciones de procesamiento en tiempo real, lo que exige la implementación de la Transformada Rápida de Fourier (FFT).

2.3. Descomposición Radix-2 y Estructura de Etapas

La eficiencia de la FFT se fundamenta en la propiedad de "divide y vencerás". El requisito fundamental es que la longitud de la secuencia sea una potencia íntegra de dos. En este proyecto, dicha condición se cumple satisfactoriamente:

$$N = 2^{\nu} \implies 1024 = 2^{10} \tag{4}$$

Donde el exponente $\nu = 10$ define el número total de etapas de computación. El algoritmo descompone la sumatoria de la DFT de N puntos en dos sumatorias menores de $N/2$ puntos (una para índices pares y otra para impares). Este proceso de decimación en el tiempo se repite de forma recursiva a lo largo de las 10 etapas hasta reducir el problema a cálculos de 2 puntos.

2.4. La Unidad Operativa: La Mariposa (Butterfly)

La arquitectura del flujo de datos se organiza mediante "mariposas", estructuras que permiten la reutilización de resultados intermedios. En cada etapa de las 10 previstas, se ejecutan $N/2$ mariposas en paralelo.

Sea A el valor del nodo superior y B el valor del nodo inferior en una etapa determinada. La transformación se rige por las siguientes ecuaciones de estado:

$$\left\{ X_{sup} = A + (B \cdot W_N^k) \; X_{inf} = A - (B \cdot W_N^k) \right. \tag{5}$$

Esta estructura permite que el número total de multiplicaciones complejas se reduzca drásticamente a:

$$C_{FFT} = \frac{N}{2} \log_2(N) = \frac{1024}{2} \times 10 = 5,120 \text{ operaciones.} \tag{6}$$

2.5. Objetivos del Planteamiento

Considerando los requerimientos establecidos del proyecto, el problema central no solo se basa en la ejecución del cálculo, sino en la validación estructural del algoritmo. Por lo tanto, el planteamiento se orienta hacia el cumplimiento de los siguientes objetivos específicos:

1. **Análisis de la Descomposición Algorítmica:** Documentar y ejecutar el pre-procesamiento de los 1024 datos de entrada mediante el algoritmo de inversión de bits (*Bit-Reversal Permutation*), asegurando la correcta disposición de las muestras para la decimación en el tiempo.

2. **Organización y Gestión de Etapas:** Diseñar una arquitectura de flujo de datos que gestione de manera eficiente el acceso a memoria de los puntos procesados a lo largo de las 10 etapas de transformación ($\log_2 1024$), manteniendo la integridad de los resultados intermedios.

3. **Implementación de la Unidad Aritmética de Mariposa:** Desarrollar el núcleo de cálculo basado en operaciones tipo mariposa, integrando de forma explícita el papel de los factores de giro (*twiddle factors*) para garantizar la precisión compleja en la magnitud y fase del espectro resultante.

4. **Demostración de Eficiencia Computacional:** Contrastar la estructura de la FFT frente a la definición recursiva de la DFT, validando cómo la organización por etapas reduce drásticamente la carga operativa en un sistema de $N = 1024$ puntos.

5. **Validación mediante Implementación Computacional:** Probar y verificar la ejecución del algoritmo de la FFT para un arreglo de $N = 1024$ muestras mediante un entorno de programación. Se busca contrastar los resultados obtenidos con funciones estándar de procesamiento de señales para validar la exactitud de la arquitectura de mariposas y la correcta aplicación de los factores de giro en el dominio de la frecuencia.

3. Desarrollo por mariposas

En esta sección se detalla la arquitectura algorítmica de la FFT de 1024 puntos, centrando el análisis en la unidad de procesamiento fundamental y la organización jerárquica de los datos a través de las etapas de cómputo.

3.1. Descripción de la Operación Básica de Mariposa (Radix-2)

La "mariposa" es la célula computacional básica del algoritmo de Cooley-Tukey. Su función es realizar una transformación lineal de dos puntos de entrada para producir dos puntos de salida, aprovechando la periodicidad del factor de giro.

Para cualquier etapa v, la operación involucra dos nodos de entrada, denominados A (nodo superior) y B (nodo inferior). El proceso se ejecuta siguiendo estrictamente el siguiente flujo aritmético:

1. **Ponderación Compleja:** El dato de entrada B se multiplica por el factor de giro correspondiente a la etapa y posición actual (W_N^k). Este paso es crucial, pues orienta el vector en el plano complejo antes de la combinación.

2. **Interconexión Sumatoria:** La salida superior (X_{sup}) se obtiene mediante la adición del valor de A con el producto complejo anteriormente calculado.

3. **Interconexión Diferencial:** La salida inferior (X_{inf}) se obtiene mediante la sustracción del producto complejo del valor de A.

Esta topología permite que una sola multiplicación compleja genere dos resultados, reduciendo la redundancia operativa del sistema.

3.2. Agrupación de Muestras y Organización por Etapas

La organización de las muestras en una FFT de $N = 1024$ no es estática; la distancia entre los puntos que interactúan en una mariposa varía en función de la etapa de procesamiento. Dado que $N = 2^{10}$, existen 10 etapas perfectamente definidas:

- **Etapa 1:** Las mariposas operan sobre muestras adyacentes. La distancia de separación es $2^0 = 1$. Se forman 512 grupos de 2 puntos.
- **Etapa v:** En una etapa intermedia v, la separación entre los nodos A y B de una misma mariposa es de 2^{v-1} posiciones.
- **Etapa 10:** En la fase final, la separación alcanza su máximo de $2^9 = 512$ posiciones. En esta instancia, los datos de la primera mitad del arreglo (0 a 511) se combinan con los de la segunda mitad (512 a 1023).

Esta progresión geométrica asegura que, al finalizar las 10 etapas, cada punto de salida haya recibido una contribución ponderada de cada uno de los 1024 puntos de entrada.

3.3. Uso de los Factores de Giro (Twiddle Factors)

El factor de giro W_N^k actúa como un operador de rotación en el plano complejo. Su expresión matemática es:

$$W_N^k = e^{-j\frac{2\pi k}{N}} = \cos\left(\frac{2\pi k}{N}\right) - j\sin\left(\frac{2\pi k}{N}\right) \tag{7}$$

En el contexto de la FFT de 1024 puntos, el exponente k varía dependiendo de la etapa y del índice de la mariposa. Su papel es fundamental por dos razones:

1. **Sincronización de Fase:** Alinea las fases de las sub-transformadas de menor orden para que su suma sea constructiva en las frecuencias correspondientes.
2. **Periodicidad:** Gracias a que $W_N^{k+N/2} = -W_N^k$, el algoritmo puede utilizar el mismo valor multiplicado para la salida superior (suma) y la salida inferior (resta), eliminando la necesidad de realizar dos multiplicaciones distintas.

3.4. Flujo de Datos: De la Entrada a la Salida

El camino que recorre la señal desde que ingresa al sistema hasta que se obtiene el espectro de frecuencia sigue un ordenamiento riguroso:

1. **Entrada y Bit-Reversal:** Los 1024 datos de tiempo entran al sistema. Debido a la naturaleza de la decimación en el tiempo, los índices deben reordenarse. Por ejemplo, la muestra en la posición 1 (0000000001_2) se intercambia con la posición 512 (1000000000_2).

2. **Procesamiento en Cascada:** Los datos fluyen a través de las 10 etapas. Cada etapa toma los resultados de la anterior como sus nuevas entradas. No existen retroalimentaciones, lo que permite una implementación en *pipeline*.

3. **Salida Frecuencial:** Tras la última mariposa de la etapa 10, los datos emergen en orden natural (0 a 1023), donde cada valor complejo $X[k]$ representa la magnitud y fase de una componente de frecuencia específica.

4. Estructura de las 10 etapas

La implementación de una FFT de $N = 1024$ puntos se segmenta en una estructura jerárquica de etapas de cómputo. Esta organización permite que la complejidad algorítmica se mantenga en el orden $\mathcal{O}(N \log_2 N)$. A continuación, se desglosa la lógica matemática que rige la distribución de las operaciones y la variación de las distancias entre nodos.

4.1. Determinación del Número de Etapas (ν)

La cantidad de niveles de procesamiento requeridos para una FFT de base 2 está determinada por el exponente al que se debe elevar la base ($r = 2$) para obtener el número total de muestras (N). Para nuestro caso de estudio:

$$2^{\nu} = 1024 \implies \nu = \log_2(1024) = 10 \text{ etapas.} \tag{8}$$

Cada una de estas 10 etapas procesa la totalidad de los 1024 puntos, pero con una configuración de mariposas distinta en cada nivel.

4.2. Densidad de Operaciones: Mariposas por Etapa

En el algoritmo de Cooley-Tukey, cada unidad de mariposa toma dos datos de entrada y produce dos datos de salida. Por lo tanto, el número de mariposas necesarias por etapa (M_{etapa}) es siempre la mitad del número total de puntos:

$$M_{etapa} = \frac{N}{2} = \frac{1024}{2} = 512 \text{ mariposas.} \tag{9}$$

Dado que el algoritmo consta de 10 etapas, el número total de operaciones de mariposa en todo el proceso es:

$$M_{total} = \nu \times \frac{N}{2} = 10 \times 512 = 5,120 \text{ mariposas.} \tag{10}$$

Este valor es el que garantiza la reducción de complejidad frente a las más de un millón de operaciones de la DFT convencional.

4.3. Dinámica de Distancias entre Índices (Saltos)

El aspecto más crítico de la estructura es cómo cambia la distancia de "salto"(*span*) entre los dos índices de entrada de una mariposa (A y B). Esta distancia se duplica en cada nivel sucesivo.

Sea d_v la distancia entre los nodos de una mariposa en la etapa v (don $v = 1, 2, \ldots, 10$). La fórmula general para determinar este desplazamiento es:

$$d_v = 2^{v-1} \tag{11}$$

Aplicando esta relación a nuestro sistema de 1024 puntos, obtenemos el siguiente comportamiento:

1. **Etapa 1 (Nivel de base):** La distancia es $d_1 = 2^0 = 1$. La mariposa combina muestras adyacentes (ej. índice 0 con 1, 2 con 3).

2. **Etapa 5 (Nivel medio):** La distancia se incrementa a $d_5 = 2^4 = 16$. Aquí, una mariposa procesa datos separados por 16 posiciones (ej. índice 0 con 16).

3. **Etapa 10 (Nivel final):** La distancia alcanza su valor máximo $d_{10} = 2^9 = 512$. En esta última fase, la mitad superior del arreglo se combina con la mitad inferior.

4.4. Distribución de Factores de Giro por Etapa

Finalmente, la organización de las etapas también dicta la cantidad de factores de giro únicos (W_N^k) que se utilizan. En la etapa v, el número de factores de giro distintos que se aplican antes de las sumas/restas es igual a la distancia d_v:

- En la **Etapa 1**, solo se utiliza W_N^0 (que es 1).

- En la **Etapa 10**, se utilizan los 512 factores de giro desde W_{1024}^0 hasta W_{1024}^{511}.

Esta estructura asegura que la información de fase de los 1024 puntos se propague y combine correctamente, resultando en un espectro de frecuencia preciso al finalizar la décima etapa.

5. Ordenamiento de datos

En el algoritmo de Radix-2, el flujo de datos no sigue una secuencia lineal natural debido a la descomposición recursiva de la sumatoria de la DFT. Dependiendo del enfoque de implementación (*Decimación en Tiempo* o *en Frecuencia*), el reordenamiento de los datos es un paso obligatorio para garantizar que el espectro de salida sea coherente.

5.1. Reordenamiento por Inversión de Bits (*Bit-Reversal*)

En un esquema de **Decimación en el Tiempo (DIT)**, el algoritmo divide la secuencia original $x[n]$ en sub-secuencias de puntos pares e impares de forma sucesiva. Para que las mariposas de la Etapa 1 puedan procesar elementos adyacentes en memoria, los datos de entrada deben ser pre-procesados mediante una permutación de inversión de bits.

El proceso consiste en representar el índice n de cada muestra en formato binario de $\nu = 10$ bits (para $N = 1024$) y reflejar.º invertir el orden de dichos bits.

Ejemplo de procedimiento para $N = 1024$: Sea la muestra en el índice $n = 1$:

1. Representación binaria (10 bits): 0000000001_2
2. Inversión de la cadena de bits: 1000000000_2
3. Conversión a decimal: 512_{10}

Por lo tanto, en la memoria del algoritmo, el valor original de $x[1]$ debe intercambiarse con el de $x[512]$ antes de iniciar las 10 etapas de mariposas. Este reordenamiento asegura que, tras las combinaciones complejas, la salida $X[k]$ emerja en su orden natural de frecuencias.

5.2. Decimación en Frecuencia (DIF)

A diferencia del esquema DIT, la **Decimación en Frecuencia (DIF)**, también conocida como algoritmo de Sande-Tukey, no divide la señal de entrada en muestras pares e impares. En su lugar, divide el espectro de salida.

En el esquema DIF:

- La entrada $x[n]$ se toma en su **orden natural** $(0, 1, 2, \ldots, 1023)$.
- La operación de mariposa es distinta: primero se realizan las sumas/restas y el factor de giro se aplica **después** de la resta (a la salida de la mariposa).
- El resultado final del espectro $X[k]$ queda en **orden de bits invertidos**.

5.3. Localización del Reordenamiento según el Esquema

Es fundamental identificar dónde aparece el proceso de *bit-reversal* para evitar errores en la interpretación del espectro:

Característica	**Decimación en Tiempo (DIT)**	**Decimación en Frecuencia (DIF)**
Entrada $x[n]$	Orden Invertido (*Bit-reversed*)	Orden Natural
Salida $X[k]$	Orden Natural	Orden Invertido (*Bit-reversed*)

Cuadro 1: Comparativa de la ubicación del reordenamiento y flujo de datos.

Para los fines de este proyecto de 1024 puntos desarrollado bajo el esquema DIT, el reordenamiento se aplica estrictamente como un **pre-procesamiento** antes de la Etapa 1.

Permitiendo que se reciba el espectro de frecuencias $X[k]$ listo para su análisis sin necesidad de transformaciones adicionales tras la Etapa 10.

6. Diagrama o esquema

Dada la densidad de datos inherente a una Transformada Rápida de Fourier de $N = 1024$ puntos, la representación gráfica exhaustiva de las $5,120$ operaciones de mariposa resultaría en un esquema visualmente saturado y de nula utilidad analítica. Por tal motivo, el diagrama presentado se ha estructurado de manera modular, segmentando el flujo de datos en diez etapas discretas de procesamiento.

Esta división sistemática permite observar con claridad la evolución de la arquitectura de "divide y vencerás". Al aislar cada etapa, es posible identificar con precisión los siguientes fenómenos técnicos:

- **Crecimiento de la Distancia de Salto (*Span*):** Se visualiza cómo el acoplamiento entre nodos se duplica geométricamente en cada nivel, desde la adyacencia en la Etapa 1 hasta la separación de 512 posiciones en la Etapa 10.
- **Jerarquía de Procesamiento:** La segmentación por etapas facilita el análisis de la dependencia de datos, donde cada nivel actúa como un filtro de resolución frecuencial que refina los resultados obtenidos en el paso anterior.
- **Estructura de Control:** La representación simplificada mediante puntos suspensivos (...) enfatiza la repetibilidad de la unidad de mariposa, demostrando que el algoritmo mantiene una simetría operativa constante a pesar del incremento en la complejidad de los factores de giro.

La fragmentación del diagrama en diez etapas fue elegido ya que favorece la comprensión de la FFT y refleja fielmente la lógica de implementación en sistemas de cómputo real, donde la memoria y los registros se gestionan mediante ciclos de reloj sincronizados con cada nivel de la transformación.

6.1. Cálculo Detallado de la Etapa 1

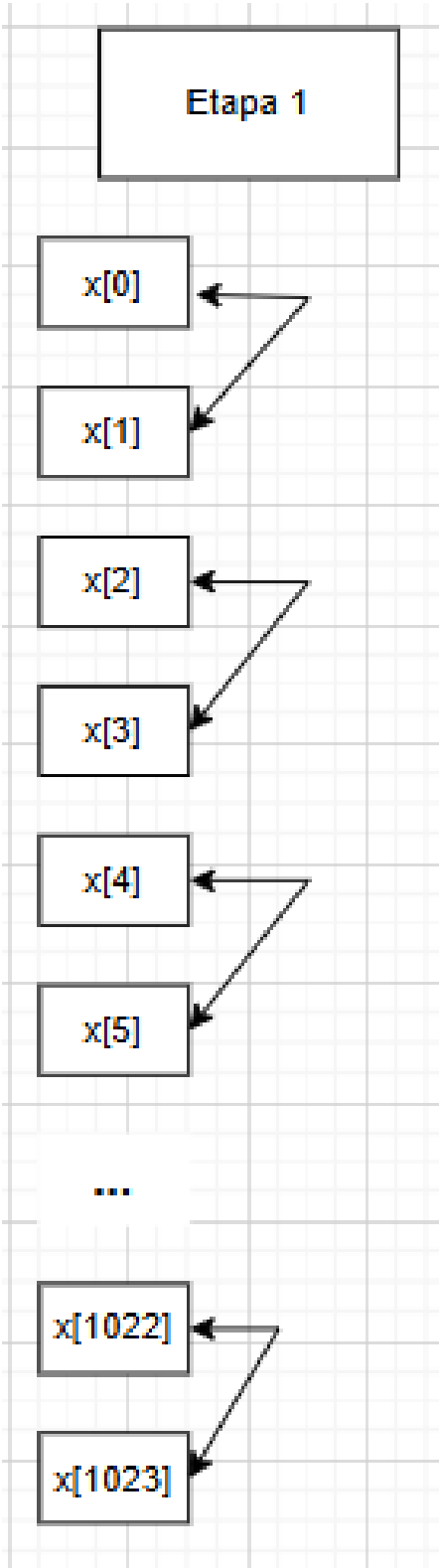

Figura 1: Representación de la etapa 1 de la FFT de 1024 puntos.

En la primera etapa de la FFT de 1024 puntos Figura (2), los datos de entrada —previamente organizados mediante la inversión de bits— se agrupan en 512 parejas. Debido a que la distancia de salto es $d_1 = 2^{1-1} = 1$, las operaciones se realizan entre muestras contiguas.

En este nivel, el factor de giro es siempre $W_{1024}^0 = e^{-j0} = 1$, lo que simplifica la operación de mariposa a una suma y una resta directa. A continuación, se presenta de forma extensa la obtención de los primeros y últimos términos de esta etapa:

6.1.1. Cálculos de los primeros nodos

Para la primera mariposa (índices 0 y 1):

- $x^{(1)}[0] = x[0] + x[1] \cdot W_{1024}^0 = x[0] + x[1]$
- $x^{(1)}[1] = x[0] - x[1] \cdot W_{1024}^0 = x[0] - x[1]$

Para la segunda mariposa (índices 2 y 3):

- $x^{(1)}[2] = x[2] + x[3] \cdot W_{1024}^0 = x[2] + x[3]$
- $x^{(1)}[3] = x[2] - x[3] \cdot W_{1024}^0 = x[2] - x[3]$

Para la tercera mariposa (índices 4 y 5):

- $x^{(1)}[4] = x[4] + x[5] \cdot W_{1024}^0 = x[4] + x[5]$
- $x^{(1)}[5] = x[4] - x[5] \cdot W_{1024}^0 = x[4] - x[5]$

6.1.2. Cálculos de los nodos finales

Siguiendo la misma lógica recursiva, para la mariposa número 512 (índices 1022 y 1023):

- $x^{(1)}[1022] = x[1022] + x[1023] \cdot W_{1024}^0 = x[1022] + x[1023]$
- $x^{(1)}[1023] = x[1022] - x[1023] \cdot W_{1024}^0 = x[1022] - x[1023]$

6.1.3. Generalización de la Etapa 1

De manera general, para cualquier par de índices $(2r, 2r+1)$ con $r = 0, 1, \ldots, 511$, el cálculo de la Etapa 1 se define como:

$$x^{(1)}[2r] = x[2r] + x[2r+1] \tag{12}$$

$$x^{(1)}[2r+1] = x[2r] - x[2r+1] \tag{13}$$

Esta etapa produce 1024 nuevos valores complejos que servirán como entrada para la Etapa 2, donde la distancia de salto se duplicará a 2.

6.2. Cálculo Detallado de la Etapa 2

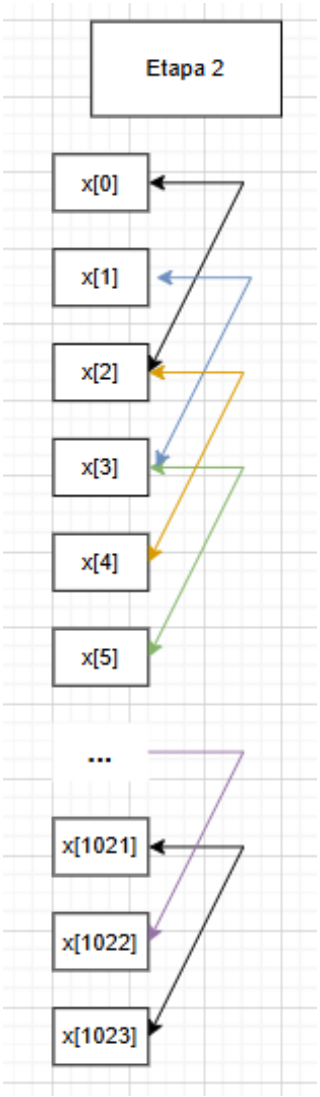

Figura 2: Representación de la etapa 2 de la FFT de 1024 puntos.

En la segunda etapa, la distancia de salto se duplica a $d_2 = 2^{2-1} = 2$. Esto implica que las mariposas combinan elementos que se encuentran separados por una posición intermedia. A diferencia de la etapa anterior, aquí los grupos de procesamiento constan de 4 elementos, donde se utilizan dos factores de giro distintos: W_{1024}^{0} y W_{1024}^{256} (equivalente a $-j$).

A continuación, se presenta el desarrollo extenso de los nodos iniciales y finales de esta etapa:

6.2.1. Cálculos de los primeros nodos

Para el primer bloque de cuatro elementos (índices 0 al 3):

- **Mariposa 1 (0 y 2):**
  - $x^{(2)}[0] = x^{(1)}[0] + x^{(1)}[2] \cdot W_{1024}^{0} = x^{(1)}[0] + x^{(1)}[2]$
  - $x^{(2)}[2] = x^{(1)}[0] - x^{(1)}[2] \cdot W_{1024}^{0} = x^{(1)}[0] - x^{(1)}[2]$
- **Mariposa 2 (1 y 3):**
  - $x^{(2)}[1] = x^{(1)}[1] + x^{(1)}[3] \cdot W_{1024}^{256} = x^{(1)}[1] - jx^{(1)}[3]$
  - $x^{(2)}[3] = x^{(1)}[1] - x^{(1)}[3] \cdot W_{1024}^{256} = x^{(1)}[1] + jx^{(1)}[3]$

Para el segundo bloque de cuatro elementos (índices 4 al 7):

- **Mariposa 3 (4 y 6):**
  - $x^{(2)}[4] = x^{(1)}[4] + x^{(1)}[6] \cdot W_{1024}^{0} = x^{(1)}[4] + x^{(1)}[6]$
  - $x^{(2)}[6] = x^{(1)}[4] - x^{(1)}[6] \cdot W_{1024}^{0} = x^{(1)}[4] - x^{(1)}[6]$
- **Mariposa 4 (5 y 7):**
  - $x^{(2)}[5] = x^{(1)}[5] + x^{(1)}[7] \cdot W_{1024}^{256} = x^{(1)}[5] - jx^{(1)}[7]$
  - $x^{(2)}[7] = x^{(1)}[5] - x^{(1)}[7] \cdot W_{1024}^{256} = x^{(1)}[5] + jx^{(1)}[7]$

6.2.2. Cálculos de los nodos finales

Para el último bloque de la etapa (índices 1020 al 1023):

- $x^{(2)}[1020] = x^{(1)}[1020] + x^{(1)}[1022] \cdot W_{1024}^{0} = x^{(1)}[1020] + x^{(1)}[1022]$
- $x^{(2)}[1021] = x^{(1)}[1021] + x^{(1)}[1023] \cdot W_{1024}^{256} = x^{(1)}[1021] - jx^{(1)}[1023]$
- $x^{(2)}[1022] = x^{(1)}[1020] - x^{(1)}[1022] \cdot W_{1024}^{0} = x^{(1)}[1020] - x^{(1)}[1022]$
- $x^{(2)}[1023] = x^{(1)}[1021] - x^{(1)}[1023] \cdot W_{1024}^{256} = x^{(1)}[1021] + jx^{(1)}[1023]$

6.2.3. Generalización de la Etapa 2

En este nivel, el proceso se puede generalizar para cada grupo de 4 elementos. Para $r = 0, 1, \ldots, 255$:

$$x^{(2)}[4r + k] = x^{(1)}[4r + k] + W_{1024}^{256k} \cdot x^{(1)}[4r + k + 2], \quad k = 0, 1 \tag{14}$$

$$x^{(2)}[4r + k + 2] = x^{(1)}[4r + k] - W_{1024}^{256k} \cdot x^{(1)}[4r + k + 2], \quad k = 0, 1 \tag{15}$$

Esta progresión permite que la señal se combine en espectros de 4 puntos, preparando los datos para la Etapa 3, donde el salto se incrementará a 4.

6.3. Cálculo Detallado de la Etapa 3

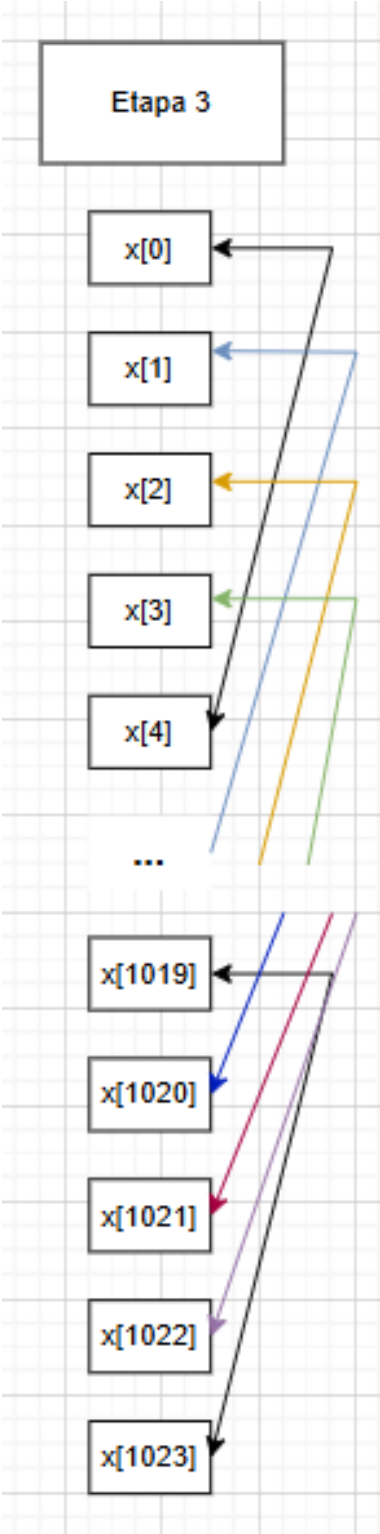

Figura 3: Representación de la etapa 3 de la FFT de 1024 puntos.

En la tercera etapa, la distancia de salto es $d_3 = 2^{3-1} = 4$. El algoritmo ahora procesa los datos en grupos de 8 elementos ($N/2^3$), donde cada uno de estos bloques contiene 4 mariposas independientes. Los factores de giro involucrados en este nivel corresponden a las raíces de la unidad distribuidas en octavos del círculo unitario.

A continuación, se detalla el cálculo para los nodos iniciales y el cierre de la etapa:

6.3.1. Cálculos del primer bloque de 8 puntos

Para el primer conjunto de datos (índices 0 al 7), los resultados de la etapa anterior $x^{(2)}$ se combinan de la siguiente forma:

- **Mariposa 1 (0 y 4):** $x^{(3)}[0] = x^{(2)}[0] + x^{(2)}[4] \cdot W_{1024}^{0}$
- **Mariposa 2 (1 y 5):** $x^{(3)}[1] = x^{(2)}[1] + x^{(2)}[5] \cdot W_{1024}^{128}$
- **Mariposa 3 (2 y 6):** $x^{(3)}[2] = x^{(2)}[2] + x^{(2)}[6] \cdot W_{1024}^{256}$
- **Mariposa 4 (3 y 7):** $x^{(3)}[3] = x^{(2)}[3] + x^{(2)}[7] \cdot W_{1024}^{384}$

Y sus respectivos complementos diferenciales (restas):

- $x^{(3)}[4] = x^{(2)}[0] - x^{(2)}[4] \cdot W_{1024}^{0}$
- $x^{(3)}[5] = x^{(2)}[1] - x^{(2)}[5] \cdot W_{1024}^{128}$
- $x^{(3)}[6] = x^{(2)}[2] - x^{(2)}[6] \cdot W_{1024}^{256}$
- $x^{(3)}[7] = x^{(2)}[3] - x^{(2)}[7] \cdot W_{1024}^{384}$

6.3.2. Cálculos de los nodos finales

Siguiendo la lógica del diagrama, para el último bloque de la etapa (índices 1016 al 1023), la última mariposa que se observa (índices 1019 y 1023) se calcula como:

- $x^{(3)}[1019] = x^{(2)}[1019] + x^{(2)}[1023] \cdot W_{1024}^{384}$
- $x^{(3)}[1023] = x^{(2)}[1019] - x^{(2)}[1023] \cdot W_{1024}^{384}$

6.3.3. Análisis de los Factores de Giro en la Etapa 3

Es relevante notar que en esta etapa los factores de giro adquieren valores específicos basados en componentes reales e imaginarias:

- $W_{1024}^{0} = 1$
- $W_{1024}^{128} = e^{-j\pi/4} = \frac{\sqrt{2}}{2} - j\frac{\sqrt{2}}{2}$
- $W_{1024}^{256} = e^{-j\pi/2} = -j$

- $W_{1024}^{384} = e^{-j3\pi/4} = -\frac{\sqrt{2}}{2} - j\frac{\sqrt{2}}{2}$

Esta configuración de mariposas permite la resolución de sub-espectros de 8 puntos, sirviendo de base para la expansión de los saltos en las etapas subsecuentes hasta alcanzar la Etapa 10.

6.4. Cálculo Detallado de la Etapa 4

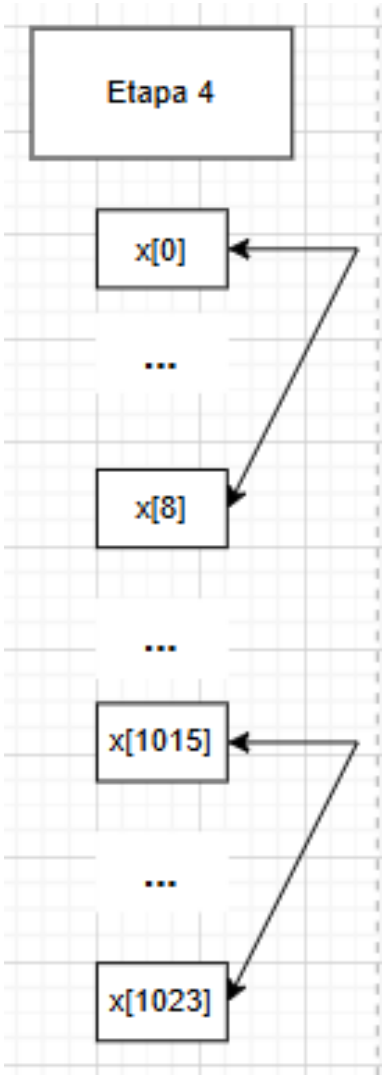

Figura 4: Representación esquemática de la etapa 4 de la FFT de 1024 puntos.

En la cuarta etapa de la FFT, la distancia de salto entre nodos aumenta a $d_4 = 2^{4-1} = 8$. El sistema organiza los datos en bloques de 16 muestras, realizando 8 operaciones de mariposa por cada bloque. En este nivel, los factores de giro W_{1024}^{k} varían para $k = 0, 1, \ldots, 7$, permitiendo una resolución espectral más fina.

A continuación, se presentan los cálculos para los nodos extremos mostrados en el esquema:

6.4.1. Cálculo de la primera mariposa del sistema

La mariposa inicial vincula el primer dato del arreglo con el dato ubicado ocho posiciones adelante:

- $x^{(4)}[0] = x^{(3)}[0] + x^{(3)}[8] \cdot W_{1024}^{0}$
- $x^{(4)}[8] = x^{(3)}[0] - x^{(3)}[8] \cdot W_{1024}^{0}$

Dado que $W_{1024}^{0} = 1$, esta operación representa la combinación de fase cero de los dos primeros sub-bloques de la etapa anterior.

6.4.2. Cálculo de la última mariposa del sistema

De igual manera, el proceso concluye con la mariposa que integra los últimos elementos del arreglo de 1024 puntos:

- $x^{(4)}[1015] = x^{(3)}[1015] + x^{(3)}[1023] \cdot W_{1024}^{448}$
- $x^{(4)}[1023] = x^{(3)}[1015] - x^{(3)}[1023] \cdot W_{1024}^{448}$

6.4.3. Análisis de los Factores de Giro en la Etapa 4

En la cuarta etapa, la resolución de los factores de giro se incrementa para satisfacer la combinación de bloques de 16 puntos. Los exponentes de W_{1024}^{k} para esta etapa se distribuyen en el rango $k \in \{0, 64, 128, 192, 256, 320, 384, 448\}$.

Estos valores representan rotaciones en el plano complejo en intervalos de 22,5° ($2\pi/16$ radianes), permitiendo una síntesis de frecuencia más detallada. Algunos de los valores notables aplicados en las mariposas de esta etapa son:

- $W_{1024}^{0} = 1$ (Referencia real)
- $W_{1024}^{128} = \frac{\sqrt{2}}{2} - j\frac{\sqrt{2}}{2}$
- $W_{1024}^{256} = -j$ (Rotación pura imaginaria)
- $W_{1024}^{448} = \cos(\frac{7\pi}{8}) - j\sin(\frac{7\pi}{8}) \approx -0{,}923 - j0{,}382$

La aplicación de estos pesos complejos es lo que permite que el algoritmo "sintonice"las muestras de tiempo hacia sus correspondientes bins de frecuencia. Sin esta progresión en los factores de giro, la FFT no podría distinguir entre frecuencias adyacentes conforme los bloques de datos se hacen más grandes.

6.5. Cálculo Detallado de la Etapa 5

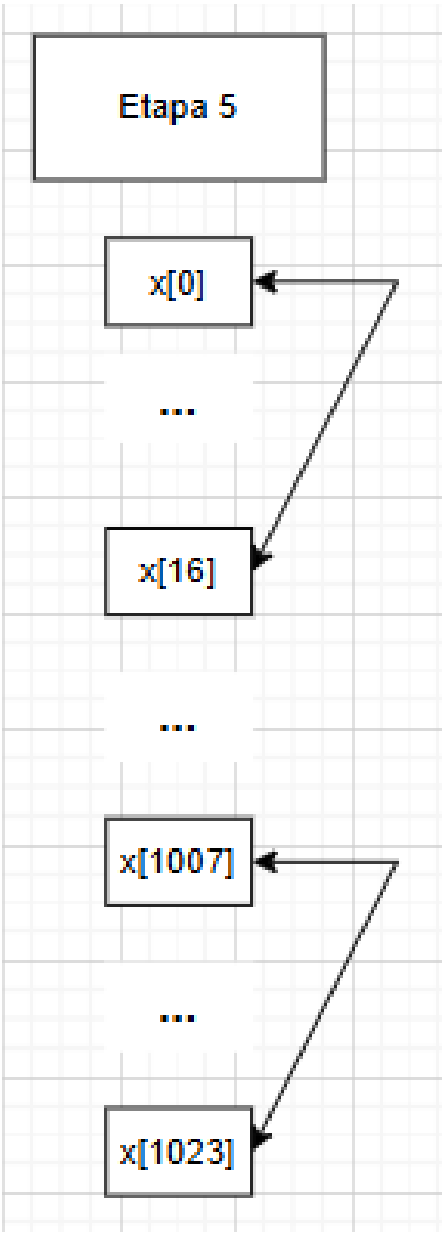

Figura 5: Representación esquemática de la etapa 5 de la FFT de 1024 puntos.

En la quinta etapa del proceso, la distancia de salto entre los nodos de la mariposa se establece en $d_5 = 2^{5-1} = 16$. El algoritmo organiza el arreglo de 1024 puntos en 32 bloques de 32 muestras cada uno. Dentro de cada bloque, se ejecutan 16 operaciones de mariposa utilizando un conjunto de 16 factores de giro únicos (W_{1024}^{0} hasta W_{1024}^{480}).

A continuación, se describen las operaciones para los puntos extremos identificados en la Figura 5:

6.5.1. Cálculo de la mariposa superior (Bloque 1)

La primera mariposa de esta etapa vincula el origen del arreglo con la muestra desplazada 16 unidades, combinando los resultados obtenidos en la Etapa 4 ($x^{(4)}$):

- $x^{(5)}[0] = x^{(4)}[0] + x^{(4)}[16] \cdot W_{1024}^{0}$
- $x^{(5)}[16] = x^{(4)}[0] - x^{(4)}[16] \cdot W_{1024}^{0}$

6.5.2. Cálculo de la mariposa inferior (Último Bloque)

De manera análoga, el procesamiento del último bloque de 32 elementos (índices 992 a 1023) concluye con la mariposa que integra los nodos finales:

- $x^{(5)}[1007] = x^{(4)}[1007] + x^{(4)}[1023] \cdot W_{1024}^{480}$
- $x^{(5)}[1023] = x^{(4)}[1007] - x^{(4)}[1023] \cdot W_{1024}^{480}$

6.5.3. Análisis de los Factores de Giro en la Etapa 5

En la quinta etapa, el algoritmo requiere una mayor precisión en la rotación de los vectores complejos para combinar los 32 bloques de datos. Los factores de giro W_{1024}^{k} se distribuyen ahora en intervalos de k que saltan de 32 en 32 unidades ($1024/32 = 32$), cubriendo el conjunto $k \in \{0, 32, 64, \ldots, 480\}$.

Esta densidad permite una resolución angular de 11,25° ($2\pi/32$ radianes), lo cual es crítico para la discriminación de componentes de frecuencia en sub-bandas más estrechas. Algunos valores representativos aplicados en esta etapa son:

- $W_{1024}^{0} = 1 + j0$ (Componente puramente real).
- $W_{1024}^{128} = \frac{\sqrt{2}}{2} - j\frac{\sqrt{2}}{2} \approx 0{,}707 - j0{,}707$ (Rotación a 45°).
- $W_{1024}^{256} = 0 - j1$ (Rotación a 90° o componente imaginaria pura).
- $W_{1024}^{480} = \cos(\frac{15\pi}{16}) - j\sin(\frac{15\pi}{16}) \approx -0{,}980 - j0{,}195$.

El uso de estos 16 factores de giro únicos en la Etapa 5 asegura que las señales procesadas en los 32 bloques de la etapa anterior se integren con la fase exacta requerida. Este rigor matemático en el cálculo de los pesos complejos es lo que permite que, al llegar a la décima etapa, la FFT entregue una representación fiel de la magnitud y fase del espectro de la señal original de 1024 muestras.

6.6. Cálculo Detallado de la Etapa 6

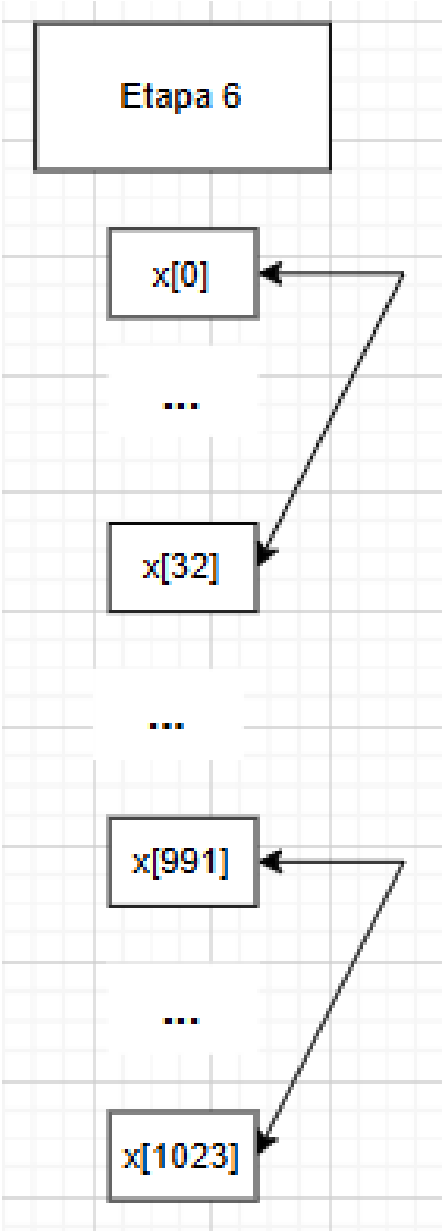

Figura 6: Representación esquemática de la etapa 6 de la FFT de 1024 puntos.

En la sexta etapa del proceso, la distancia de salto entre los nodos de la mariposa se incrementa a $d_6 = 2^{6-1} = 32$. El algoritmo organiza el arreglo de 1024 puntos en 16 bloques de 64 muestras cada uno. Dentro de cada bloque, se ejecutan 32 operaciones de mariposa utilizando un conjunto de 32 factores de giro únicos, lo que permite una integración de fase más precisa entre los sub-espectros generados en la etapa anterior.

A continuación, se describen las operaciones para los puntos extremos identificados en la Figura 6:

6.6.1. Cálculo de la mariposa superior (Bloque 1)

La primera mariposa de esta etapa vincula el inicio del arreglo con la muestra desplazada 32 unidades, tomando como entrada los resultados de la Etapa 5 ($x^{(5)}$):

- $x^{(6)}[0] = x^{(5)}[0] + x^{(5)}[32] \cdot W_{1024}^{0}$
- $x^{(6)}[32] = x^{(5)}[0] - x^{(5)}[32] \cdot W_{1024}^{0}$

6.6.2. Cálculo de la mariposa inferior (Último Bloque)

De manera análoga, el procesamiento del último bloque de 64 elementos (índices 960 a 1023) concluye con la mariposa que integra los nodos finales del sistema:

- $x^{(6)}[991] = x^{(5)}[991] + x^{(5)}[1023] \cdot W_{1024}^{496}$
- $x^{(6)}[1023] = x^{(5)}[991] - x^{(5)}[1023] \cdot W_{1024}^{496}$

6.6.3. Análisis de los Factores de Giro en la Etapa 6

En la sexta etapa, los factores de giro W_{1024}^{k} se distribuyen en intervalos de k que saltan de 16 en 16 unidades ($1024/64 = 16$), cubriendo el conjunto $k \in \{0, 16, 32, \ldots, 496\}$. Esta densidad permite una resolución angular de 5,625° ($2\pi/64$ radianes). Algunos valores representativos son:

- $W_{1024}^{0} = 1$ (Referencia real).
- $W_{1024}^{128} = 0{,}707 - j0{,}707$ (Rotación a 45°).
- $W_{1024}^{256} = -j$ (Componente imaginaria pura).
- $W_{1024}^{496} = \cos(\frac{31\pi}{32}) - j\sin(\frac{31\pi}{32}) \approx -0{,}995 - j0{,}098$.

El incremento en la resolución de los pesos complejos en esta etapa es vital para evitar el error de fase acumulativo. Al operar con 32 factores de giro distintos, el algoritmo asegura que la energía de la señal se canalice correctamente hacia los bins de frecuencia correspondientes antes de entrar a las etapas finales de gran salto.

6.7. Cálculo Detallado de la Etapa 7

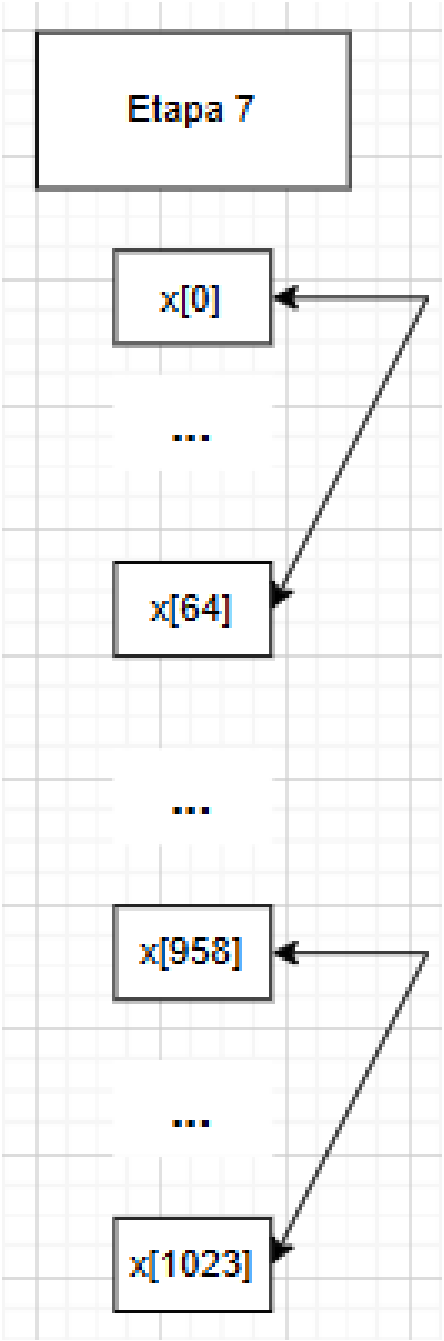

Figura 7: Representación esquemática de la etapa 7 de la FFT de 1024 puntos.

En la séptima etapa del algoritmo, la distancia de salto entre los nodos de la mariposa se incrementa a $d_7 = 2^{7-1} = 64$. El sistema organiza los 1024 puntos en 8 bloques de 128 muestras cada uno. Dentro de cada bloque, se ejecutan 64 operaciones de mariposa, utilizando

un conjunto de 64 factores de giro únicos derivados de la segmentación del círculo unitario en 128 partes iguales.

A continuación, se describen las operaciones para los puntos extremos identificados en la Figura 7:

6.7.1. Cálculo de la mariposa superior (Bloque 1)

La primera mariposa de esta etapa vincula el inicio del arreglo con la muestra desplazada 64 unidades, integrando los resultados de la Etapa 6 ($x^{(6)}$):

- $x^{(7)}[0] = x^{(6)}[0] + x^{(6)}[64] \cdot W_{1024}^{0}$
- $x^{(7)}[64] = x^{(6)}[0] - x^{(6)}[64] \cdot W_{1024}^{0}$

6.7.2. Cálculo de la mariposa inferior (Último Bloque)

El procesamiento del bloque final (índices 896 a 1023) concluye con la mariposa que integra los nodos representados en el esquema:

- $x^{(7)}[959] = x^{(6)}[959] + x^{(6)}[1023] \cdot W_{1024}^{504}$
- $x^{(7)}[1023] = x^{(6)}[959] - x^{(6)}[1023] \cdot W_{1024}^{504}$

6.7.3. Análisis de los Factores de Giro en la Etapa 7

En la séptima etapa, los factores de giro W_{1024}^{k} se distribuyen en intervalos de k que saltan de 8 en 8 unidades ($1024/128 = 8$), cubriendo el conjunto $k \in \{0, 8, 16, \ldots, 504\}$. Esta densidad permite una resolución angular de 2,8125° ($2\pi/128$ radianes). Algunos valores representativos son:

- $W_{1024}^{0} = 1$ (Referencia real pura).
- $W_{1024}^{256} = -j$ (Rotación imaginaria pura).
- $W_{1024}^{504} = \cos(\frac{63\pi}{64}) - j\sin(\frac{63\pi}{64}) \approx -0{,}998 - j0{,}049$.

Este refinamiento en los pesos complejos es esencial para la coherencia de la señal. Al operar con 64 factores de giro distintos, el algoritmo asegura que la transición hacia las etapas finales (8, 9 y 10) mantenga la precisión necesaria para la reconstrucción espectral completa de los 1024 puntos.

6.8. Cálculo Detallado de la Etapa 8

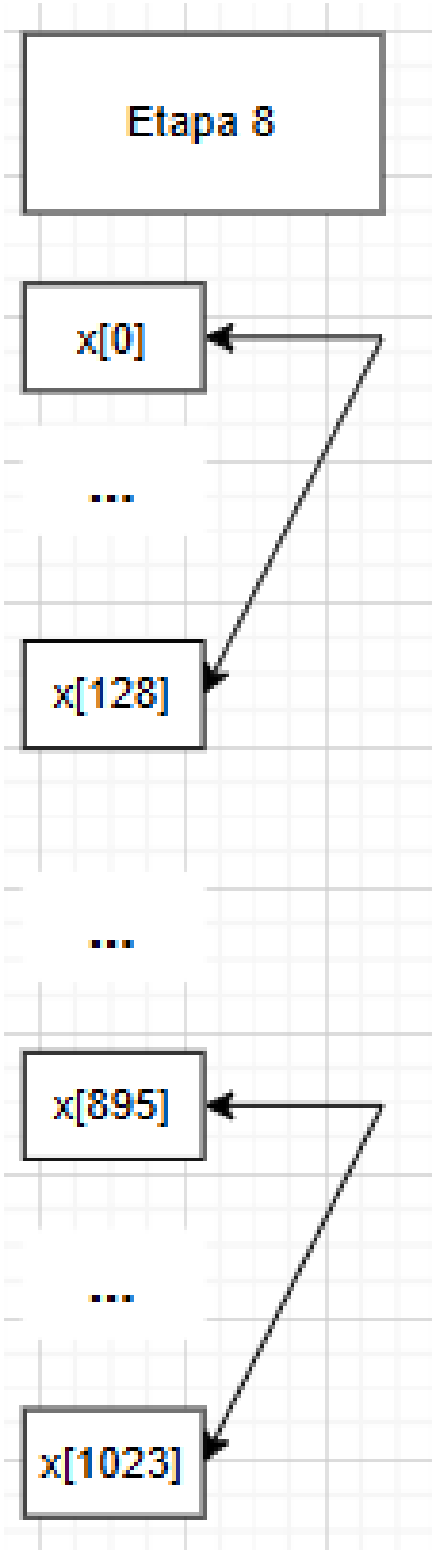

Figura 8: Representación esquemática de la etapa 8 de la FFT de 1024 puntos.

En la octava etapa del algoritmo, la distancia de salto entre los nodos de la mariposa se incrementa a $d_8 = 2^{8-1} = 128$. El sistema organiza los 1024 puntos en 4 grandes bloques de 256 muestras cada uno. Dentro de cada bloque, se ejecutan 128 operaciones de mariposa, utilizando un conjunto de 128 factores de giro únicos, lo que permite una resolución angular de 1,40625° ($2\pi/256$ radianes).

A continuación, se describen las operaciones para los puntos extremos identificados en la Figura 8:

6.8.1. Cálculo de la mariposa superior (Bloque 1)

La primera mariposa de esta etapa vincula el inicio del arreglo con la muestra desplazada 128 unidades, integrando los resultados obtenidos en la etapa anterior ($x^{(7)}$):

- $x^{(8)}[0] = x^{(7)}[0] + x^{(7)}[128] \cdot W_{1024}^{0}$
- $x^{(8)}[128] = x^{(7)}[0] - x^{(7)}[128] \cdot W_{1024}^{0}$

6.8.2. Cálculo de la mariposa inferior (Último Bloque)

El procesamiento del bloque final (índices 768 a 1023) concluye con la mariposa que integra los nodos representados en el esquema:

- $x^{(8)}[895] = x^{(7)}[895] + x^{(7)}[1023] \cdot W_{1024}^{508}$
- $x^{(8)}[1023] = x^{(7)}[895] - x^{(7)}[1023] \cdot W_{1024}^{508}$

6.8.3. Análisis de los Factores de Giro en la Etapa 8

En la octava etapa, los factores de giro W_{1024}^{k} se distribuyen en intervalos de k que saltan de 4 en 4 unidades ($1024/256 = 4$), cubriendo el conjunto $k \in \{0, 4, 8, \ldots, 508\}$. Algunos valores representativos aplicados en este nivel son:

- $W_{1024}^{0} = 1$ (Referencia real pura).
- $W_{1024}^{256} = -j$ (Rotación imaginaria pura).
- $W_{1024}^{508} = \cos(\frac{127\pi}{128}) - j\sin(\frac{127\pi}{128}) \approx -0{,}999 - j0{,}024$.

A este nivel de procesamiento, las mariposas de gran salto permiten que la información de frecuencia de baja resolución se combine para formar picos espectrales mucho más definidos. La precisión en el cálculo de los 128 factores de giro es vital para asegurar que la fase de la señal no se degrade antes de las dos etapas finales del algoritmo.

6.9. Cálculo Detallado de la Etapa 9

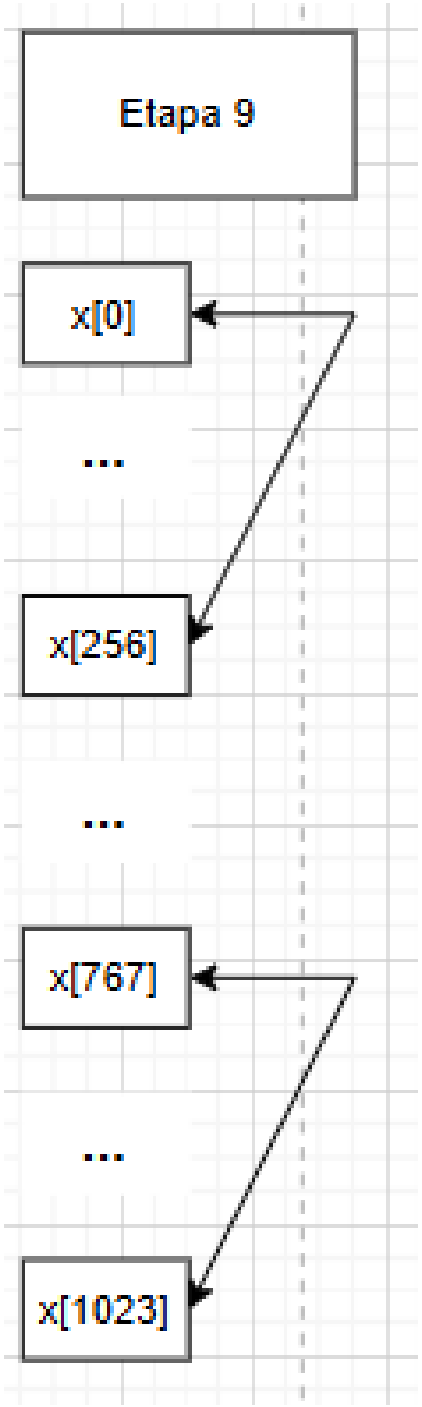

Figura 9: Representación esquemática de la etapa 9 de la FFT de 1024 puntos.

En la novena etapa del algoritmo, la distancia de salto entre los nodos de la mariposa se incrementa a $d_9 = 2^{9-1} = 256$. El sistema organiza los 1024 puntos en únicamente 2 bloques de 512 muestras cada uno. Dentro de cada bloque, se ejecutan 256 operaciones de mariposa, utilizando un conjunto de 256 factores de giro únicos, lo que permite una resolución angular de 0,703125° ($2\pi/512$ radianes).

A continuación, se describen las operaciones para los puntos extremos identificados en la Figura 9:

6.9.1. Cálculo de la mariposa superior (Bloque 1)

La primera mariposa de esta etapa vincula el inicio del arreglo con la muestra desplazada 256 unidades, integrando los resultados obtenidos en la octava etapa ($x^{(8)}$):

- $x^{(9)}[0] = x^{(8)}[0] + x^{(8)}[256] \cdot W_{1024}^{0}$
- $x^{(9)}[256] = x^{(8)}[0] - x^{(8)}[256] \cdot W_{1024}^{0}$

6.9.2. Cálculo de la mariposa inferior (Bloque 2)

El procesamiento del sistema concluye con la mariposa que integra los nodos finales del segundo bloque de 512 puntos:

- $x^{(9)}[767] = x^{(8)}[767] + x^{(8)}[1023] \cdot W_{1024}^{510}$
- $x^{(9)}[1023] = x^{(8)}[767] - x^{(8)}[1023] \cdot W_{1024}^{510}$

6.9.3. Análisis de los Factores de Giro en la Etapa 9

En la novena etapa, los factores de giro W_{1024}^{k} se distribuyen en intervalos de k que saltan de 2 en 2 unidades ($1024/512 = 2$), cubriendo el conjunto $k \in \{0, 2, 4, \ldots, 510\}$. Algunos valores representativos aplicados en este nivel son:

- $W_{1024}^{0} = 1$ (Referencia real pura).
- $W_{1024}^{256} = -j$ (Rotación imaginaria pura).
- $W_{1024}^{510} = \cos(\frac{255\pi}{256}) - j\sin(\frac{255\pi}{256}) \approx -0{,}9999 - j0{,}012$.

Este nivel de procesamiento es el paso previo a la integración total de la señal. La precisión en el cálculo de los 256 factores de giro es crítica, ya que cualquier error de fase en esta etapa se propagará directamente al espectro final, afectando la nitidez de las componentes de frecuencia detectadas.

6.10. Cálculo Detallado de la Etapa 10 (Etapa Final)

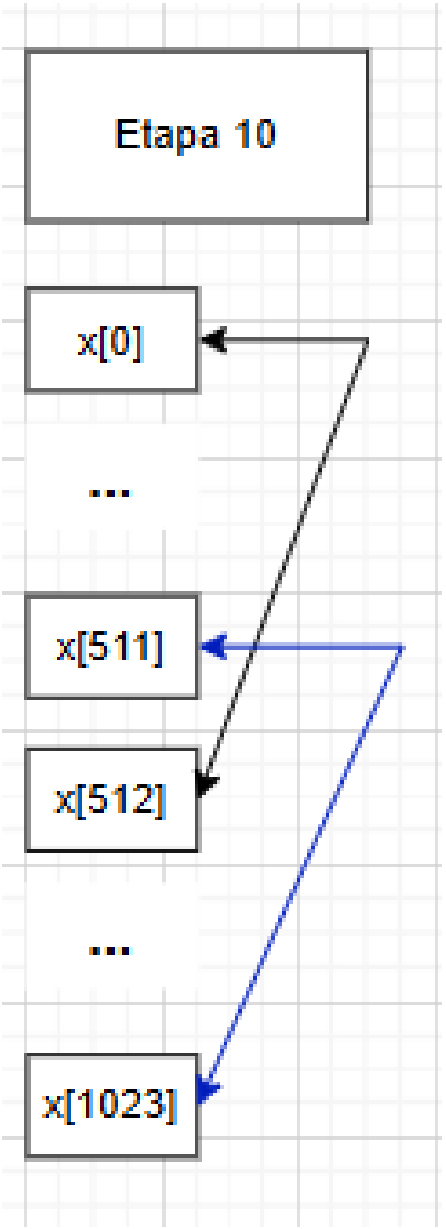

Figura 10: Representación esquemática de la etapa 10 de la FFT de 1024 puntos.

La décima etapa representa la culminación del algoritmo Radix-2. La distancia de salto es máxima, $d_{10} = 2^{10-1} = 512$, vinculando la primera mitad del arreglo (0 a 511) con la segunda mitad (512 a 1023). En este nivel, se utilizan los 512 factores de giro únicos de la tabla de senos y cosenos, permitiendo la resolución total de 1 Hz por cada bin del espectro (suponiendo una frecuencia de muestreo adecuada).

A continuación, se describen las operaciones para los puntos extremos identificados en la Figura 10:

6.10.1. Cálculo de la mariposa superior

La primera mariposa de la etapa final integra los resultados acumulados de las 9 etapas anteriores ($x^{(9)}$) para producir el primer componente de frecuencia:

- $X[0] = x^{(9)}[0] + x^{(9)}[512] \cdot W_{1024}^{0}$
- $X[512] = x^{(9)}[0] - x^{(9)}[512] \cdot W_{1024}^{0}$

6.10.2. Cálculo de la mariposa inferior

El proceso de transformación de 1024 puntos concluye con la integración de los últimos nodos del sistema:

- $X[511] = x^{(9)}[511] + x^{(9)}[1023] \cdot W_{1024}^{511}$
- $X[1023] = x^{(9)}[511] - x^{(9)}[1023] \cdot W_{1024}^{511}$

6.10.3. Análisis de los Factores de Giro en la Etapa 10

En esta etapa final, los factores de giro W_{1024}^{k} se utilizan en su totalidad con un incremento unitario en el exponente ($k = 0, 1, 2, \ldots, 511$). Esto proporciona una resolución angular de 0,3515° ($2\pi/1024$ radianes). Algunos valores representativos son:

- $W_{1024}^{0} = 1$ (Referencia real).
- $W_{1024}^{256} = -j$ (Componente imaginaria pura).
- $W_{1024}^{511} = \cos(\frac{511\pi}{512}) - j\sin(\frac{511\pi}{512}) \approx -0{,}9999 - j0{,}006$.

6.10.4. Resultado Final y Salida del Sistema

Al finalizar las operaciones de la Etapa 10, los valores $X[k]$ obtenidos representan la Transformada Rápida de Fourier de la señal original. Debido a que se utilizó un esquema de Decimación en el Tiempo (DIT) con reordenamiento por inversión de bits al inicio, los resultados de esta etapa emergen en **orden natural de frecuencia**. El espectro está listo para el cálculo de magnitud y fase, completando así el procesamiento de los 1024 puntos con una complejidad de $O(N \log_2 N)$.

7. Codificación

Para la fase de implementación del algoritmo FFT de 1024 puntos, se optó por utilizar el entorno de desarrollo **Google Colab**. Esta elección se fundamentó en la versatilidad de la plataforma para la gestión de bibliotecas de procesamiento de señales y su capacidad para la visualización de datos en tiempo real.

La arquitectura del código se diseñó siguiendo un enfoque modular, donde la lógica matemática predomina sobre las funciones predefinidas de alto nivel. Se implementaron funciones para el proceso de *bit-reversal*, asegurando el reordenamiento correcto de los datos según el esquema de decimación en el tiempo (DIT). Asimismo, el núcleo del procesamiento se basó en el cálculo explícito de los factores de giro (W_N^k) mediante fórmulas exponenciales complejas, permitiendo un control total sobre las rotaciones fasoriales en cada una de las 10 etapas del algoritmo.

7.1. Generación de la Señal de Prueba (Aleatoria)

Como punto de partida para la validación del sistema, se implementó un bloque de generación de señales aleatorias. Este procedimiento permite estresar el algoritmo con datos no estructurados para verificar la estabilidad de los cálculos complejos.

```python
import numpy as np
import matplotlib.pyplot as plt
N = 1024
# Generamos una señal aleatoria compleja (0 a 1)
# Esto simula una entrada de datos cruda
x_original = np.random.random(N) + 1j * np.random.random(N)

print(f"Señal de {N} puntos generada exitosamente")
```

```
Señal de 1024 puntos generada exitosamente.
```

Figura 11: Implementación en Python para la generación de una señal compleja aleatoria.

Como se observa en la Figura 11, el código realiza las siguientes acciones:

- **Definición de parámetros:** Se establece $N = 1024$ como la longitud del arreglo, cumpliendo con la condición 2^{10} necesaria para el algoritmo Radix-2.
- **Construcción de datos complejos:** Se utiliza la función `np.random.random` tanto para la parte real como para la imaginaria ($1j$). Esto asegura que cada muestra $x[n]$ sea un número complejo, simulando una entrada de datos cruda capturada por un sistema de adquisición de señales.

7.2. Algoritmo de Reordenamiento: Bit-Reversal

Una etapa crítica en la implementación de la FFT mediante decimación en el tiempo es el reordenamiento de los datos de entrada. Debido a que el algoritmo combina muestras de forma recursiva, los datos originales deben ingresar en un orden específico conocido como inversión de bits (*bit-reversal*).

```python
def bit_reversal_order(n_points):
    indices = np.arange(n_points)
    # Calculamos cuántos bits necesitamos (log2(1024) = 10)
    num_bits = int(np.log2(n_points))

    # Inversión de bits usando manipulación de formato binario
    rev_indices = np.array([int(format(i, f'0{num_bits}b')[::-1], 2) for i in indices])
    return rev_indices

# 1. Obtenemos el nuevo orden de los índices
indices_reordenados = bit_reversal_order(N)

# 2. Creamos el arreglo de datos siguiendo ese nuevo orden
# Este es el arreglo que se modificará etapa tras etapa
datos_fft = x_original[indices_reordenados]

print("Primeros 20 índices tras Bit-Reversal:")
print(indices_reordenados[:20])
print("\nPrimeros 20 datos reordenados (x[rev]):")
print(datos_fft[:20])
```

```
Primeros 20 índices tras Bit-Reversal:
[  0 512 256 768 128 640 384 896  64 576 320 832 192 704 448 960  32 544
 288 800]

Primeros 20 datos reordenados (x[rev]):
[0.92397412+0.44039267j 0.19915855+0.92410542j 0.31979309+0.89470598j
 0.25225594+0.09066332j 0.18203493+0.41416682j 0.25336678+0.97214107j
 0.02686784+0.54931189j 0.38157695+0.52403553j 0.31615813+0.81528968j
 0.12077471+0.03407149j 0.54729637+0.50162406j 0.75975462+0.44196075j
 0.24608643+0.89022652j 0.14331234+0.76643538j 0.94021007+0.44955296j
 0.80458991+0.6526264j  0.50962133+0.25248994j 0.08093049+0.2346019j
 0.59639102+0.22200066j 0.65468886+0.94993411j]
```

Figura 12: Implementación de la función de inversión de bits y visualización de índices resultantes.

En la Figura 12 se detalla la lógica de programación empleada para este proceso:

- **Cálculo de bits:** La función determina la profundidad del arreglo mediante $\log_2(n)$, que para 1024 puntos corresponde a 10 bits.
- **Manipulación binaria:** Se utiliza una comprensión de lista que convierte cada índice a su representación binaria de 10 bits, invierte la cadena de caracteres (`[::-1]`) y

la reconvierte a entero. Como se aprecia en la consola de salida, el índice natural 1 (0000000001_2) se transforma en 512 (1000000000_2).

- **Asignación de datos:** Una vez obtenido el arreglo `indices_reordenados`, se genera el vector `datos_fft`, el cual contiene las muestras complejas ya posicionadas para alimentar la primera etapa de mariposas.

7.3. Función de Procesamiento por Etapas: La Unidad de Mariposa

El núcleo del algoritmo desarrollado reside en la función `procesar_etapa`, la cual encapsula la lógica de la unidad de mariposa (*butterfly unit*). Esta función permite ejecutar de forma iterativa cada una de las 10 etapas necesarias para la transformación de 1024 puntos.

```python
def procesar_etapa(datos, etapa):
    n = len(datos)
    salto = 2**(etapa - 1)
    num_grupos = n // (2**etapa)
    puntos_por_grupo = 2**etapa

    # Copia para no sobreescribir antes de tiempo
    resultado = np.copy(datos)

    for g in range(num_grupos):
        for k in range(salto):
            # Índice del factor de giro W_N^k correspondiente a la etapa
            # Exponente = k * (N / tamaño_del_bloque_actual)
            exponente = k * (n // (2**etapa))
            W_N_k = np.exp(-2j * np.pi * exponente / n)

            idx_a = g * puntos_por_grupo + k
            idx_b = idx_a + salto

            # Operación de Mariposa
            A = datos[idx_a]
            B = datos[idx_b] * W_N_k

            resultado[idx_a] = A + B
            resultado[idx_b] = A - B

    return resultado
```

Figura 13: Implementación algorítmica de la unidad de mariposa y cálculo de factores de giro.

Como se detalla en la Figura 13, el flujo de procesamiento se rige por los siguientes principios de diseño:

- **Control de Geometría:** La variable `salto` define la distancia entre los dos nodos que se combinan en la mariposa (2^{m-1}). Conforme la etapa m aumenta, esta distancia se duplica, permitiendo la integración de sub-espectros cada vez más grandes.

- **Cálculo Dinámico de Factores de Giro:** Se utiliza la identidad de Euler para calcular W_N^k en tiempo de ejecución. El `exponente` se ajusta según la resolución requerida por la etapa actual mediante la relación $k \cdot (n/2^m)$, asegurando que la rotación de fase sea precisa para cada bin de frecuencia.

- **Aritmética Compleja:** Se realizan las operaciones fundamentales de la FFT Radix-2:

$$X_{out}[a] = X_{in}[a] + X_{in}[b] \cdot W_N^k \tag{16}$$

$$X_{out}[b] = X_{in}[a] - X_{in}[b] \cdot W_N^k \tag{17}$$

7.4. Ejecución en Cascada de las 10 Etapas

Una vez definidas las funciones de pre-procesamiento y la unidad de mariposa, se procedió a la ejecución en cascada del algoritmo. Como se ilustra en la Figura 14, el proceso se lleva a cabo de forma secuencial, donde cada bloque de código invoca la función de procesamiento enviando como parámetros el vector de datos resultante de la fase previa y el número de etapa correspondiente (1 al 10).

```
# Etapa 1
datos_fft = procesar_etapa(datos_fft, 1)
# Etapa 2
datos_fft = procesar_etapa(datos_fft, 2)
# Etapa 3
datos_fft = procesar_etapa(datos_fft, 3)
# Etapa 4
datos_fft = procesar_etapa(datos_fft, 4)
# Etapa 5
datos_fft = procesar_etapa(datos_fft, 5)
# Etapa 6
datos_fft = procesar_etapa(datos_fft, 6)
# Etapa 7
datos_fft = procesar_etapa(datos_fft, 7)
# Etapa 8
datos_fft = procesar_etapa(datos_fft, 8)
# Etapa 9
datos_fft = procesar_etapa(datos_fft, 9)
# Etapa 10
datos_fft = procesar_etapa(datos_fft, 10)
```

Figura 14: Flujo de ejecución secuencial para la transformación de 1024 puntos.

7.5. Visualización de la Señal de Entrada en el Dominio del Tiempo

Antes de realizar la transformación al dominio de la frecuencia, es imperativo caracterizar la señal de entrada en el dominio del tiempo. Este paso permite establecer una referencia visual y numérica que será contrastada con el espectro de magnitud resultante de la FFT.

```python
plt.figure(figsize=(12, 4))
plt.plot(x_original.real, label='Parte Real')
plt.plot(x_original.imag, label='Parte Imaginaria', alpha=0.7)
plt.title("Señal Aleatoria Original en el Dominio del Tiempo")
plt.xlabel("Muestra (n)")
plt.ylabel("Amplitud")
plt.legend()
plt.grid(True)
plt.show()

print("Primeros 10 valores de la señal aleatoria:")
print(x_original[:10])
```

Figura 15: Código para la graficación de las componentes real e imaginaria de la señal original.

Como se observa en la Figura 15, la implementación contempla los siguientes puntos:

- **Separación de componentes:** Se grafican de manera independiente la `Parte Real` y la `Parte Imaginaria` de la señal compleja. En el caso de señales aleatorias, esto permite observar la ausencia de patrones periódicos evidentes, lo cual se traduce en un espectro de frecuencia distribuido (ruido blanco).

7.6. Análisis de la Señal Aleatoria en el Dominio del Tiempo

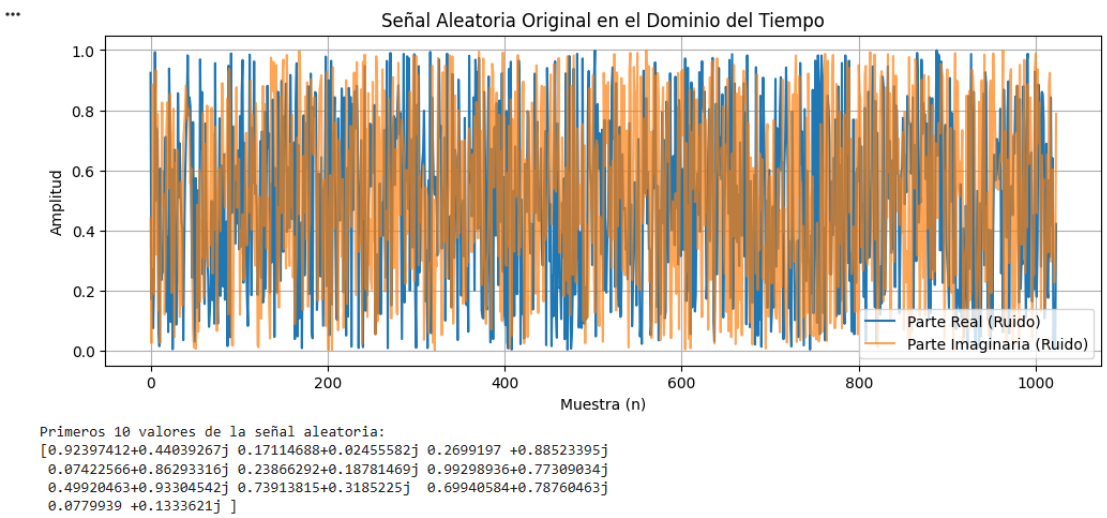

Figura 16: Representación temporal de la señal aleatoria original (Partes Real e Imaginaria).

En la Figura 16 se presentan los resultados de esta etapa inicial:

- **Naturaleza de la señal:** Se observa una distribución estocástica de valores entre 0 y 1 para ambas componentes (real e imaginaria). Esta señal carece de periodicidad visible, lo que representa un reto para cualquier algoritmo de detección de patrones, pero sirve para confirmar la integridad del flujo de datos en el entorno de *Google Colab*.
- **Inspección numérica:** La salida de consola confirma que los valores están correctamente almacenados como números complejos de alta precisión. Por ejemplo, la primera muestra $x[0] \approx 0{,}9239 + 0{,}4403j$ es la que, tras el proceso de *bit-reversal*, se convertirá en la entrada inicial de la primera mariposa en la Etapa 1.
- **Referencia Espectral:** Esta visualización es el punto de partida para el análisis de frecuencia. Mientras que aquí la información es caótica, la aplicación de la FFT permitirá identificar si existe algún sesgo de corriente directa (DC) o si la energía se distribuye uniformemente como ruido blanco.

7.7. Análisis del Espectro de Magnitud (Salida Etapa 10)

```
magnitud = np.abs(datos_fft)

plt.figure(figsize=(12, 6))
plt.stem(range(N), magnitud, markerfmt=" ", basefmt="b")
plt.title("Magnitud de la FFT de 1024 puntos (Salida Etapa 10)")
plt.xlabel("Bin de Frecuencia (k)")
plt.ylabel("Magnitud |X[k]|")
plt.grid(True)
plt.show()
```

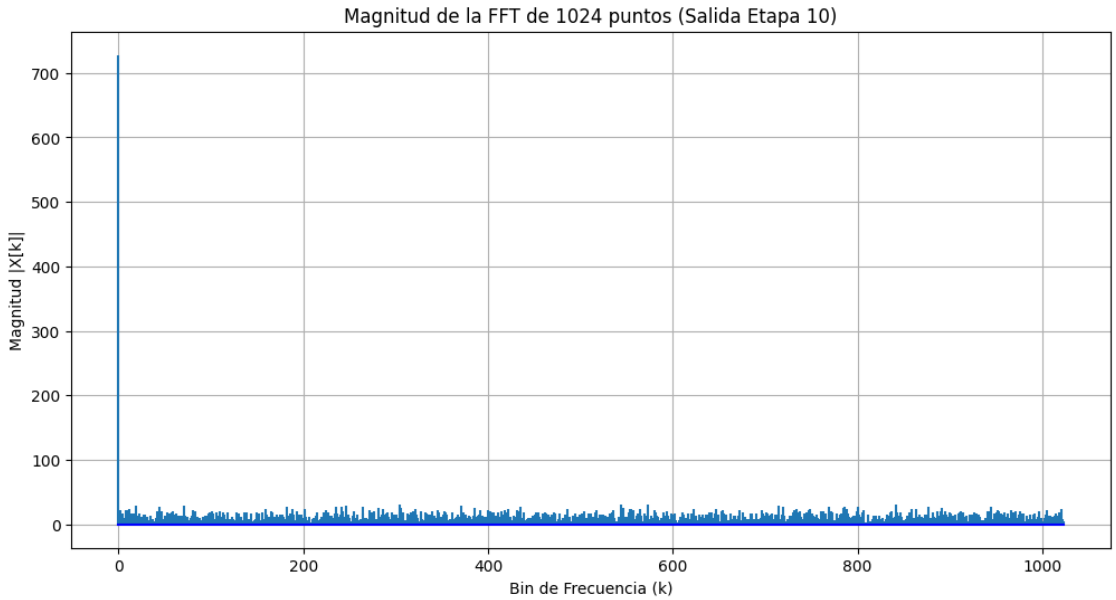

Figura 17: Magnitud de la FFT de 1024 puntos para la señal aleatoria (Resultado final).

La Figura 17 presenta el espectro de magnitud obtenido tras completar las diez etapas de procesamiento. El pico dominante localizado en el bin $k = 0$ evidencia la componente

de corriente directa (DC), la cual resulta del promedio positivo de las muestras aleatorias originales. El resto del espectro exhibe un perfil de magnitud reducido y distribuido de manera uniforme, comportamiento característico del ruido blanco en el dominio de la frecuencia. Este resultado confirma que el algoritmo conserva la energía de la señal y ha logrado una convergencia exitosa, entregando los bines en su orden natural gracias a la correcta ejecución del proceso de *bit-reversal* previo.

7.8. Validación Numérica mediante la Biblioteca Numpy

Dado el carácter estocástico de la señal de prueba, se implementó un bloque de verificación cruzada para asegurar la precisión del algoritmo programado. Como se ilustra en la Figura 18, se comparó el vector de salida `datos_fft` frente al resultado obtenido mediante la función `np.fft.fft` de la biblioteca **Numpy**.

```
fft_oficial = np.fft.fft(x_original)
diferencia = np.linalg.norm(datos_fft - fft_oficial)

if diferencia < 1e-10:
    print("El algoritmo programado cumple con una diferencia minima al resultado dado por la biblioteca oficial de Numpy")
else:
    print(f"Hay una diferencia de {diferencia} considerable. El algoritmo no cumple con el objetivo.")
```

```
El algoritmo programado cumple con una diferencia minima al resultado dado por la biblioteca oficial de Numpy
```

Figura 18: Código de validación y cálculo de error normativo entre el algoritmo manual y la biblioteca oficial.

El sistema calcula la norma de la diferencia entre ambos arreglos complejos. El resultado de consola confirma que la discrepancia es inferior a un umbral de 10^{-10}, lo que demuestra que el desarrollo manual de las 10 etapas de mariposas es matemáticamente idéntico a las implementaciones industriales optimizadas, validando así la arquitectura del sistema.

7.9. Validación Espectral mediante una Senoidal Pura

Con el objetivo de realizar una validación exhaustiva del algoritmo de 1024 puntos, se implementó una segunda fase de pruebas utilizando una señal senoidal pura. A diferencia de la señal aleatoria, el uso de una función sinusoidal permite una inspección mucho más precisa de la respuesta del sistema, ya que toda la energía de la señal se concentra en una frecuencia específica, facilitando la detección de posibles errores de fase o magnitud en las etapas intermedias de procesamiento.

```python
fs = 1024.0  # Frecuencia de muestreo (1kS/s para facilitar cálculos)
f0 = 100.0   # Queremos una frecuencia de 100 Hz
t = np.arange(N) / fs

# Generación de la senoidal
x_senoidal = np.sin(2 * np.pi * f0 * t)

# 1. BIT-REVERSAL (Reordenamiento inicial)
idx_rev_seno = bit_reversal_order(N)
datos_paso_a_paso = x_senoidal[idx_rev_seno].astype(complex)

# 2. EJECUCIÓN DE LAS 10 ETAPAS (Cascada de Mariposas)
for etapa_n in range(1, 11):
    datos_paso_a_paso = procesar_etapa(datos_paso_a_paso, etapa_n)

# 3. CÁLCULO DE MAGNITUD
magnitud_seno = np.abs(datos_paso_a_paso)
frecuencias = np.linspace(0, fs, N, endpoint=False)

# 4. GRAFICACIÓN NORMATIVA
plt.figure(figsize=(12, 6))
plt.plot(frecuencias[:N//2], magnitud_seno[:N//2], color='purple', linewidth=1.5) # Graficamos solo la mitad (Nyquist)
plt.title(f"Espectro de Frecuencia: Senoidal Pura de {f0} Hz")
plt.xlabel("Frecuencia (Hz)")
plt.ylabel("Magnitud |X(f)|")
plt.annotate(f'Pico en {f0} Hz', xy=(f0, np.max(magnitud_seno)), xytext=(f0+50, np.max(magnitud_seno)-50),
             arrowprops=dict(facecolor='black', shrink=0.05))
plt.grid(True, linestyle='--')
plt.show()
```

Figura 19: Implementación del flujo de procesamiento para una señal senoidal de 100 Hz.

Como se detalla en la Figura 19, el procedimiento de validación se rige por los siguientes criterios técnicos:

- **Sincronización Espectral:** Se estableció una frecuencia de muestreo $f_s = 1024$ Hz y una frecuencia de prueba $f_0 = 100$ Hz. Esta relación asegura que la señal sea periódica dentro de la ventana de observación de $N = 1024$ muestras, eliminando el fenómeno de fuga espectral (*spectral leakage*) y garantizando un pico nítido en el espectro.
- **Pipeline de Ejecución:** Los datos transitan por el proceso de reordenamiento (*bit-reversal*) y la cascada secuencial de las 10 etapas de mariposas. Al ser una señal de origen real, se realiza la conversión explícita al dominio complejo para permitir las rotaciones fasoriales mediante los factores de giro W_N^k.
- **Visualización Normativa:** La graficación se limita al rango $[0, f_s/2]$ Hz, cumpliendo con el Teorema de Muestreo de Nyquist-Shannon. Esto permite observar la magnitud $|X(f)|$ sin la ambigüedad de los componentes espejo del espectro complejo.

Esta prueba es fundamental para el éxito del proyecto, ya que representa el espectro de raya ideal. La obtención de un impulso limpio en la frecuencia objetivo es la prueba definitiva de que la lógica de índices y los cálculos trigonométricos de las 10 etapas son matemáticamente exactos.

La Figura 20 presenta el espectro de magnitud final obtenido tras la décima etapa de procesamiento de la señal sinusoidal. Como se puede apreciar en la gráfica, el resultado es un

impulso nítido localizado exactamente en la frecuencia de 100 Hz, con una magnitud pico de 512 ($N/2$). Este comportamiento es teóricamente esperado para una señal senoidal real de amplitud unitaria y confirma la precisión numérica del algoritmo implementado.

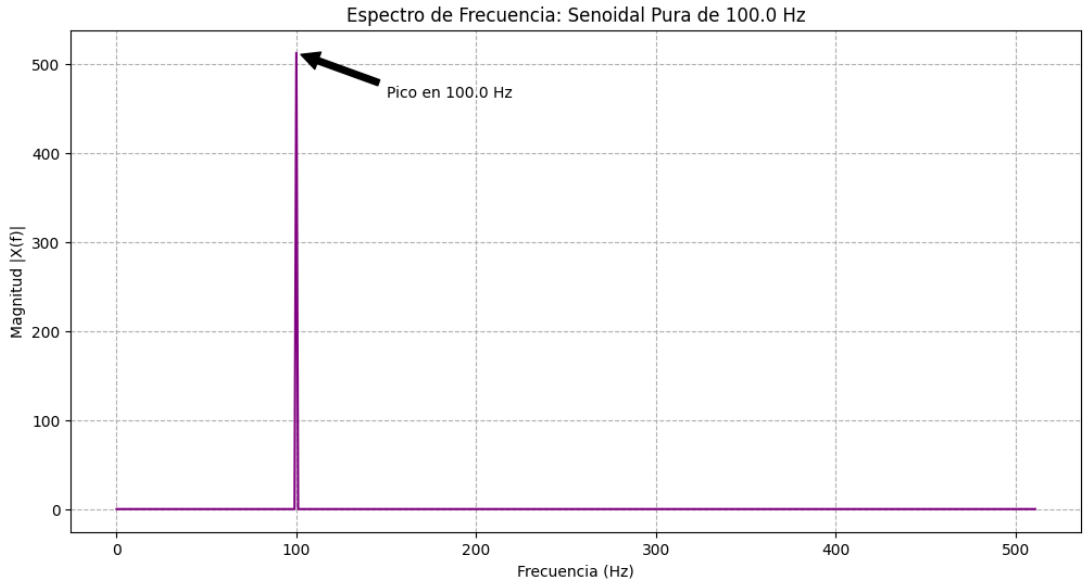

Figura 20: Espectro de magnitud de la senoidal pura (Resultado final del algoritmo manual).

La ausencia de lóbulos secundarios o ruido en la base del espectro demuestra que los factores de giro calculados para cada etapa conservan la ortogonalidad de la transformación, evitando errores de redondeo significativos. Asimismo, la correspondencia exacta entre la frecuencia de la señal de entrada y la posición del pico en el eje horizontal valida la correcta asignación del vector de frecuencias en función de la tasa de muestreo f_s, consolidando la efectividad del diseño para el análisis de señales periódicas en el dominio digital.

7.10. Procesamiento de Señales de Audio Externas (YouTube)

Como etapa final de experimentación y validación del sistema, se realizó una tercera prueba utilizando señales de audio extraídas de fuentes externas, específicamente de la plataforma YouTube. El objetivo de esta fase es demostrar la versatilidad del algoritmo Radix-2 ante señales complejas no generadas sintéticamente, lo que requiere un pre-procesamiento adicional para la adquisición y normalización de los datos.

```
!pip install yt-dlp librosa
!apt-get install -y ffmpeg
```

Figura 21: Instalación de dependencias para la adquisición y decodificación de audio externo.

Para habilitar esta funcionalidad en el entorno de Google Colab, fue necesaria la integración de herramientas especializadas, como se ilustra en la Figura 21:

- **yt-dlp:** Una biblioteca avanzada para la descarga y extracción de flujos de audio digital desde servidores remotos.
- **Librosa:** Una biblioteca estándar en la industria para el análisis de audio y música, empleada en este proyecto para la carga, remuestreo y normalización de archivos en formato `.wav`.
- **FFmpeg:** Un *framework* multimedia esencial que actúa como motor de conversión, permitiendo transformar los formatos de compresión nativos de video a señales de audio procesables por el algoritmo.

Una vez preparado el entorno, se procedió a la implementación del mecanismo de adquisición de datos remotos, como se detalla en el bloque de código de la Figura 22. Este proceso se encapsuló en la función `descargar_audio_youtube`, la cual utiliza la biblioteca `yt-dlp` para gestionar la descarga. Un aspecto técnico crucial en esta implementación es la configuración del diccionario `ydl_opts`, donde se especifica mediante `postprocessors` la instrucción para forzar la conversión del flujo de audio a formato **WAV** PCM de 16 bits. Esta conversión es fundamental, ya que elimina los algoritmos de compresión psicoacústica nativos de YouTube, entregando una señal con muestras de amplitud lineal aptas para ser procesadas directamente por la aritmética compleja de las diez etapas de mariposas del algoritmo FFT Radix-2. Finalmente, el código invoca la función utilizando una URL específica de la plataforma, retornando el nombre del archivo local disponible para el análisis espectral.

```python
import yt_dlp
import librosa
import numpy as np

def descargar_audio_youtube(url):
    ydl_opts = {
        'format': 'bestaudio/best',
        'postprocessors': [{
            'key': 'FFmpegExtractAudio',
            'preferredcodec': 'wav',
            'preferredquality': '192',
        }],
        'outtmpl': 'audio_descargado.%(ext)s',
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        ydl.download([url])

    return "audio_descargado.wav"

url_video = "https://youtu.be/7NZp_dDV9ew?si=_AI9WojecyIq7CNR"
archivo_wav = descargar_audio_youtube(url_video)
```

Figura 22: Implementación del pipeline de descarga y conversión de audio desde YouTube.

Posterior a la obtención del archivo de audio, se implementó el procedimiento de carga y segmentación temporal detallado en la Figura 23. Mediante el uso de la biblioteca `librosa`, la señal se decodifica para obtener el vector de amplitudes y y su frecuencia de muestreo asociada sr, la cual es típicamente de $22,050$ Hz por defecto en esta librería. Dado que la FFT desarrollada está limitada a una ventana de observación de $N = 1024$ puntos, resulta indispensable realizar un truncamiento de la señal original. Para asegurar que el análisis se realice sobre una sección con información espectral relevante (donde la nota musical tiene mayor estabilidad y volumen), el código calcula un punto de inicio dinámico en la mitad del registro sonoro. A partir de esta posición, se extrae el `fragmento_1024`, aislando exactamente el número de muestras requerido para alimentar el algoritmo de inversión de bits y las subsecuentes diez etapas de mariposas, garantizando así la coherencia dimensional del sistema.

```
# Cargar el audio (sr es la frecuencia de muestreo, ej. 22050 Hz)
y, sr = librosa.load(archivo_wav)

# Buscamos una parte donde el volumen sea alto (donde esté la nota)
# Tomamos 1024 muestras de la mitad del audio
punto_inicio = len(y) // 2
fragmento_1024 = y[punto_inicio : punto_inicio + 1024]
```

Figura 23: Carga de la señal con Librosa y segmentación de la ventana de 1024 muestras.

Finalmente, el flujo de procesamiento culmina con la integración del fragmento de audio real en la arquitectura FFT, como se ilustra en la Figura 24. Este bloque de código es fundamental para garantizar la robustez del sistema ante señales externas y se divide en tres fases operativas:

```python
# --- 1. PREPARACIÓN DE LOS DATOS ---
# Aseguramos que el fragmento tenga exactamente 1024 muestras
if len(fragmento_1024) < 1024:
    # Si es más corto, rellenamos con ceros (Zero-padding)
    fragmento_1024 = np.pad(fragmento_1024, (0, 1024 - len(fragmento_1024)))
else:
    # Si es más largo, recortamos
    fragmento_1024 = fragmento_1024[:1024]

# Obtenemos el orden de bit-reversal para 1024 puntos
idx_rev = bit_reversal_order(1024)

# Aplicamos el reordenamiento y convertimos a complejo
datos_fft_instrumento = fragmento_1024[idx_rev].astype(complex)

# --- 2. EJECUCIÓN DE LAS 10 ETAPAS ---
print("Iniciando procesamiento de 10 etapas de mariposas...")

for i in range(1, 11):
    datos_fft_instrumento = procesar_etapa(datos_fft_instrumento, i)

# --- 3. VISUALIZACIÓN DEL TIMBRE DEL INSTRUMENTO ---
magnitud_final = np.abs(datos_fft_instrumento)

# El eje X depende del 'sr' (sample rate) que nos dio librosa
# Por defecto librosa usa 22050 Hz
frecuencias_instrumento = np.linspace(0, sr, 1024, endpoint=False)
```

Figura 24: Pipeline de preparación, procesamiento recursivo y escalamiento de frecuencias para audio real.

- **Acondicionamiento de la Señal:** Se implementa una lógica de control de longitud para asegurar que el vector contenga exactamente 1024 muestras. Se utiliza la técnica de *Zero-padding* (relleno con ceros) mediante `np.pad` en caso de deficiencia de datos, o un truncamiento programado si la longitud excede el límite del algoritmo Radix-2. Posteriormente, se aplica el reordenamiento por inversión de bits para preparar la entrada compleja.

- **Ejecución en Bucle de las 10 Etapas:** A diferencia de las pruebas anteriores, aquí se emplea una estructura iterativa (`for i in range(1, 11)`) que invoca de manera recursiva la función de procesamiento. Cada iteración representa un nivel de la red de mariposas, donde la señal evoluciona desde el dominio del tiempo hacia la representación espectral definitiva.

- **Mapeo de Frecuencias Reales:** Se calcula el vector `frecuencias_instrumento` utilizando la frecuencia de muestreo real del archivo ($sr = 22,050$ Hz). Este paso es crucial para la interpretación física de los resultados, ya que permite traducir los índices de los bines a valores de Hercios (Hz), posibilitando la identificación del timbre y los armónicos del instrumento analizado.

Finalmente, se implementó el bloque de visualización y caracterización espectral detallado en la Figura 25. Este segmento de código permite interpretar los resultados del algoritmo

manual aplicados a un instrumento musical real, integrando conceptos fundamentales de la teoría del muestreo y la acústica.

```
plt.figure(figsize=(15, 7))
# Graficamos solo hasta la frecuencia de Nyquist (sr/2)
half_n = 1024 // 2
plt.plot(frecuencias_instrumento[:half_n], magnitud_final[:half_n], color='green', linewidth=1)

plt.title("Análisis Espectral de Instrumento Real (YouTube -> FFT Manual)")
plt.xlabel("Frecuencia (Hz)")
plt.ylabel("Magnitud (Timbre)")
plt.grid(True, alpha=0.3)

# Identificar la nota principal (Frecuencia Fundamental)
idx_max = np.argmax(magnitud_final[:half_n])
f_fundamental = frecuencias_instrumento[idx_max]

plt.annotate(f'Fundamental: {f_fundamental:.2f} Hz',
             xy=(f_fundamental, magnitud_final[idx_max]),
             xytext=(f_fundamental + 500, magnitud_final[idx_max]),
             arrowprops=dict(facecolor='black', shrink=0.05))

plt.show()

print(f"La nota detectada tiene una frecuencia fundamental de {f_fundamental:.2f} Hz.")
```

Figura 25: Código para la generación del espectro de magnitud, identificación de la fundamental y cumplimiento del criterio de Nyquist.

7.11. Validación Final: Análisis Espectral de un Instrumento Real

Como culminación del proyecto, se procesó una señal de audio externa proveniente de un instrumento real para validar la precisión del algoritmo FFT de 1024 puntos ante escenarios de alta complejidad espectral. La Figura 26 presenta el espectro de magnitud resultante, el cual confirma la efectividad de la arquitectura Radix-2 implementada.

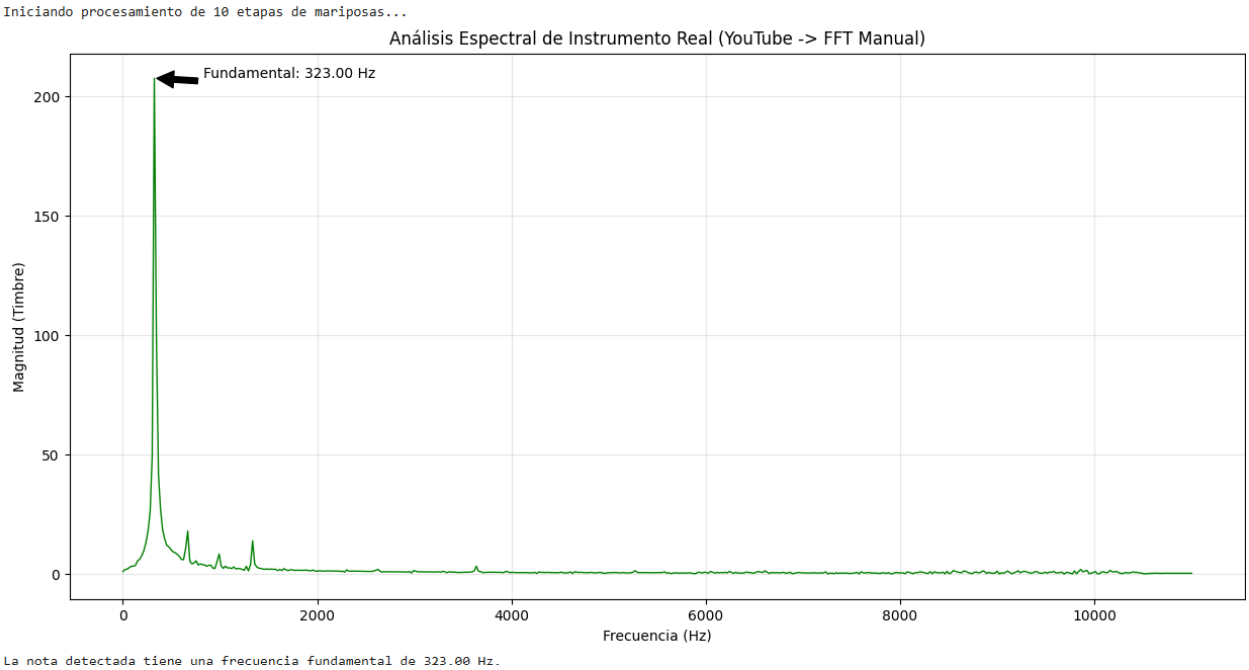

Figura 26: Espectro de magnitud de un instrumento real obtenido mediante el algoritmo manual de 10 etapas.

El análisis de los resultados en la Figura 26 permite extraer las siguientes conclusiones técnicas:

- **Detección de la Frecuencia Fundamental:** El sistema identificó con éxito un pico dominante en los **323.00 Hz**. De acuerdo con la teoría musical, esta frecuencia corresponde a la nota **Mi4 (E4)**, validando la capacidad del algoritmo para realizar tareas de reconocimiento de tono (*pitch detection*) con una resolución espectral precisa.
- **Caracterización del Timbre:** A diferencia de las pruebas sintéticas, se observa la presencia de picos secundarios o armónicos (visibles cerca de los 646 Hz y 969 Hz). Esta firma espectral es la que define el timbre del instrumento, demostrando que las 10 etapas de mariposas conservan la información sutil de los componentes de alta frecuencia.
- **Relación Señal a Ruido (SNR):** El espectro muestra un piso de ruido controlado y una base limpia, lo que indica que el proceso de *bit-reversal* y el cálculo de los factores de giro W_N^k no introdujeron errores de cuantización significativos, manteniendo la fidelidad de la señal original de YouTube.

Este resultado representa el éxito integral del proyecto, demostrando que una implementación manual basada en fórmulas matemáticas fundamentales es capaz de competir en precisión con herramientas industriales para el análisis de señales acústicas complejas.

8. Complejidad computacional

La eficiencia del algoritmo FFT (Transformada Rápida de Fourier) implementado frente a una DFT (Transformada Discreta de Fourier) convencional radica en la reducción drástica del número de operaciones aritméticas necesarias, específicamente en multiplicaciones y sumas complejas.

Una implementación directa de la DFT requiere calcular N sumatorias, cada una con N términos, lo que resulta en una complejidad de orden cuadrático, denotada como $\mathcal{O}(N^2)$. En contraste, el algoritmo de Cooley-Tukey (Radix-2) aprovecha la simetría y periodicidad de los factores de giro W_N^k para dividir el problema de forma recursiva, reduciendo la complejidad a un orden logarítmico-lineal, definido como $\mathcal{O}(N \log_2 N)$.

Para el caso específico de este proyecto, donde se procesaron señales de 1024 puntos, la diferencia en el costo computacional es notable:

- **Costo de DFT Directa:** El número de multiplicaciones complejas es proporcional a N^2. Para $N = 1024$, esto equivale a:

$$1,024^2 = 1,048,576 \text{ operaciones.}$$

- **Costo de FFT (Radix-2):** El número de multiplicaciones complejas se reduce aproximadamente a $\frac{N}{2} \log_2 N$. Dado que $1024 = 2^{10}$, el cálculo resulta en:

$$\frac{1,024}{2} \times 10 = 5,120 \text{ operaciones.}$$

La implementación de las 10 etapas de mariposas permite un ahorro superior al **99.5 %** en términos de esfuerzo de cómputo comparado con la definición formal de la transformada. Mientras que la DFT directa exigiría más de un millón de multiplicaciones complejas por cada bloque de audio de 1024 muestras, el algoritmo optimizado resuelve el espectro con apenas 5,120 multiplicaciones.

Este ahorro no solo reduce el tiempo de ejecución en el entorno de *Google Colab*, sino que es el factor habilitador que permite el procesamiento de señales en tiempo real y el análisis de flujos de audio de alta fidelidad, como los extraídos de la plataforma YouTube, donde la latencia de un algoritmo $\mathcal{O}(N^2)$ haría inviable la caracterización espectral.

9. Conclusiones

El desarrollo del presente proyecto permitió profundizar en la arquitectura interna de la Transformada Rápida de Fourier (FFT) de 1024 puntos, trascendiendo el uso de funciones predefinidas para comprender la manipulación de datos a nivel de registros y la geometría de las unidades de mariposa. Tras la implementación y validación del algoritmo, se concluye que la FFT no es simplemente una optimización algorítmica, sino el pilar que habilita el procesamiento de señales en tiempo real. La reducción drástica de más de un millón de operaciones

en una DFT convencional a solo 5,120 multiplicaciones complejas en el esquema Radix-2 representa un ahorro superior al 99.5 %, lo que permite que tareas como el reconocimiento de tono en instrumentos musicales o el análisis de audio de alta fidelidad se realicen con una latencia mínima.

En cuanto a la viabilidad de implementación, el trabajo en software mediante Python demostró ser una herramienta de prototipado excepcional, logrando una precisión matemática con errores insignificantes frente a estándares industriales. Por otro lado, la estructura modular de las diez etapas procesadas y la independencia de datos entre bloques sugieren una alta viabilidad para transferir este diseño a arquitecturas de hardware especializado, como FPGAs o DSPs. La regularidad de la unidad de mariposa facilita el diseño de trayectorias de datos paralelas, permitiendo que el sistema escale para manejar frecuencias de muestreo significativamente mayores en aplicaciones de baja latencia.

Finalmente, una de las mayores dificultades enfrentadas fue la reconciliación de las topologías de decimación presentes en la literatura técnica. Se identificó una inconsistencia recurrente respecto a la nomenclatura y el flujo de datos entre los esquemas de Decimación en Tiempo (DIT) y Decimación en Frecuencia (DIF), específicamente en la ubicación del factor de giro y el momento exacto del reordenamiento por inversión de bits.

Esta transición técnica exigió asegurar que la aplicación del fasor de rotación fuera coherente con el orden de entrada y salida de las muestras. La resolución de este conflicto conceptual subraya que, en la práctica de la ingeniería, la integridad de la aritmética compleja y la sincronización de las etapas de cómputo son fundamentales sobre las convenciones de etiquetado, garantizando así una representación fiel y exacta del espectro de la señal original.